

## Lecture 05:

## Unsupervised Learning



Markus Hohle

University of California, Berkeley

**Machine Learning Algorithms**

MSSE 277B, 3 Units



Lecture 1: Course Overview and Introduction to Machine Learning

Lecture 2: Bayesian Methods in Machine Learning

**classic ML tools & algorithms**

Lecture 3: Dimensionality Reduction: Principal Component Analysis

Lecture 4: Linear and Non-linear Regression and Classification

**Lecture 5: Unsupervised Learning: K-Means, GMM, Trees**

Lecture 6: Adaptive Learning and Gradient Descent Optimization Algorithms

Lecture 7: Introduction to Artificial Neural Networks - The Perceptron

**ANNs/AI/Deep Learning**

Lecture 8: Introduction to Artificial Neural Networks - Building Multiple Dense Layers

Lecture 9: Convolutional Neural Networks (CNNs) - Part I

Lecture 10: CNNs - Part II

Lecture 11: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs)

Lecture 12: Combining LSTMs and CNNs

Lecture 13: Running Models on GPUs and Parallel Processing

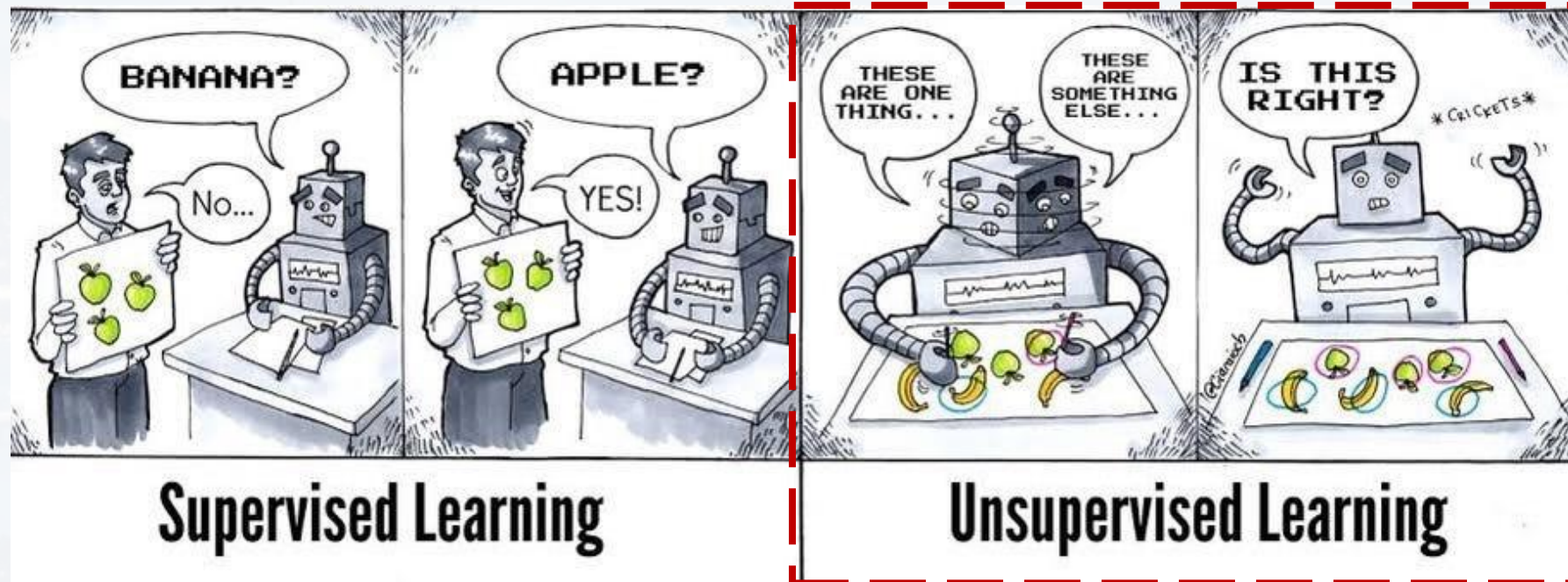
Lecture 14: Transformer

Lecture 15: GNN

Lecture 16: Project Presentations

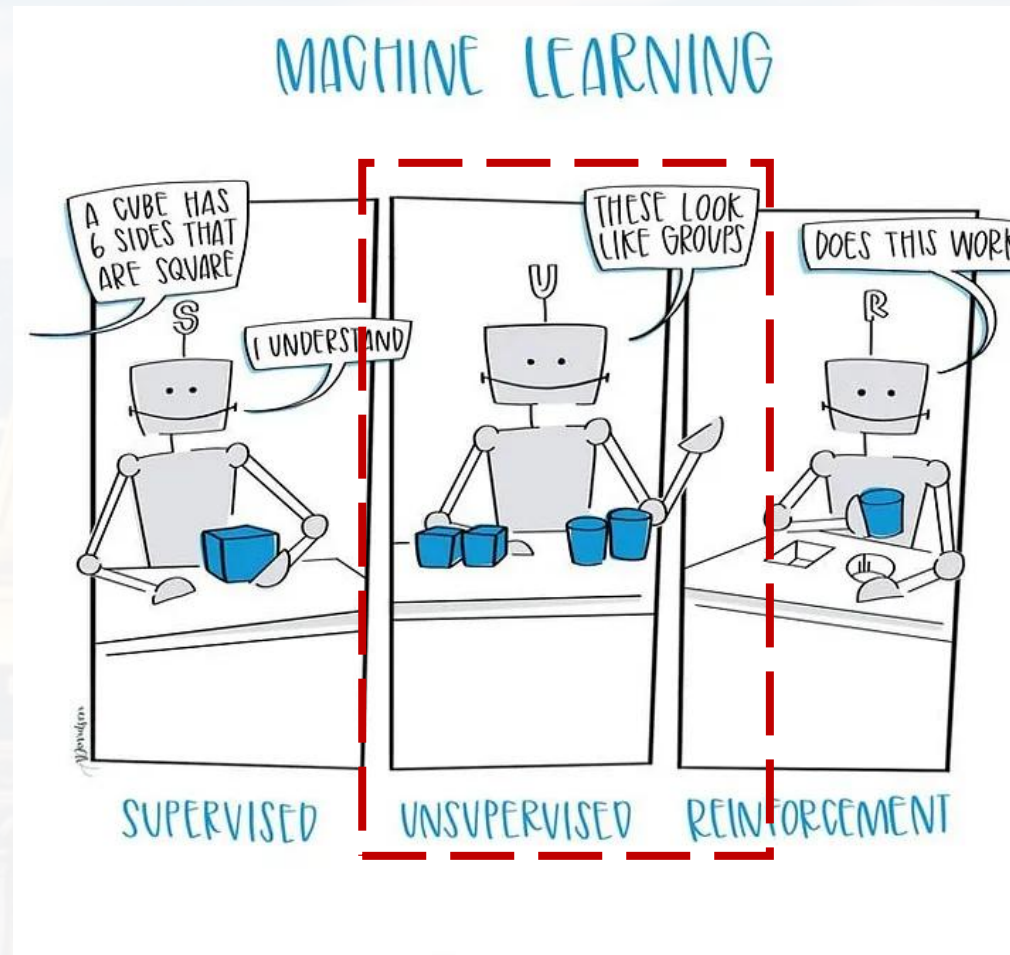
So far, there has been a **training** data set and a **test** data set...

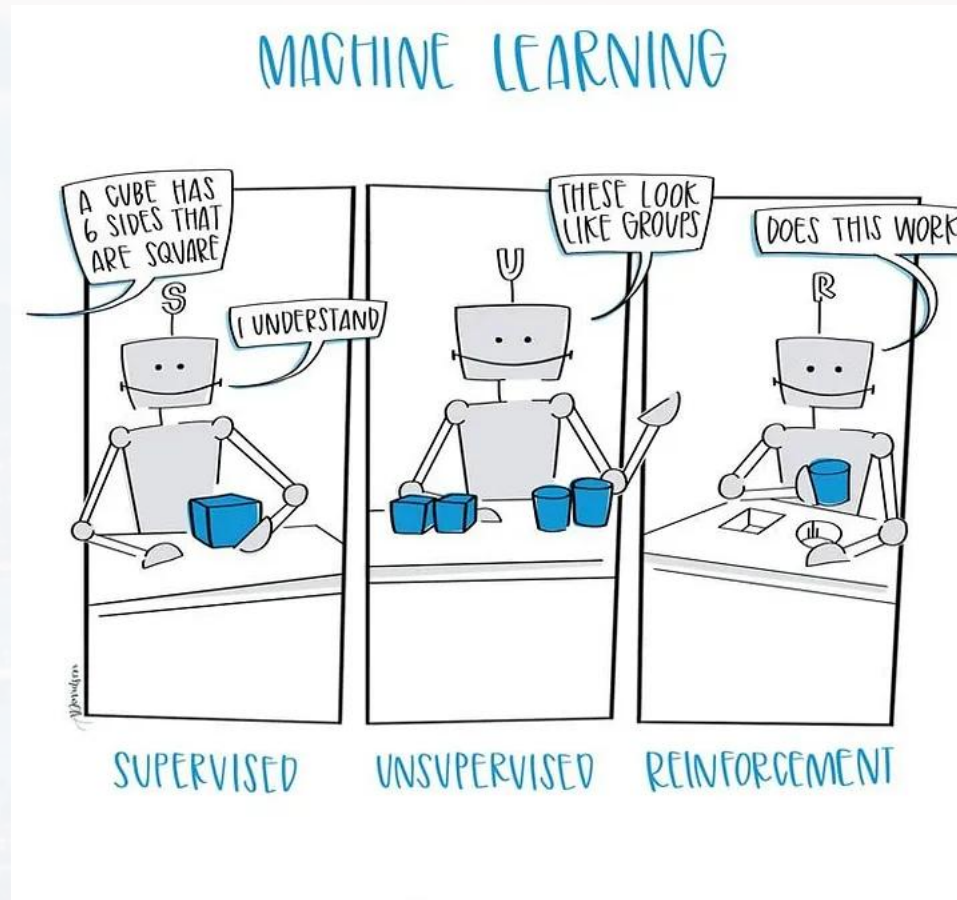
... but maybe there are ways to learn *without* training data



So far, there has been a **training** data set and a **test** data set...

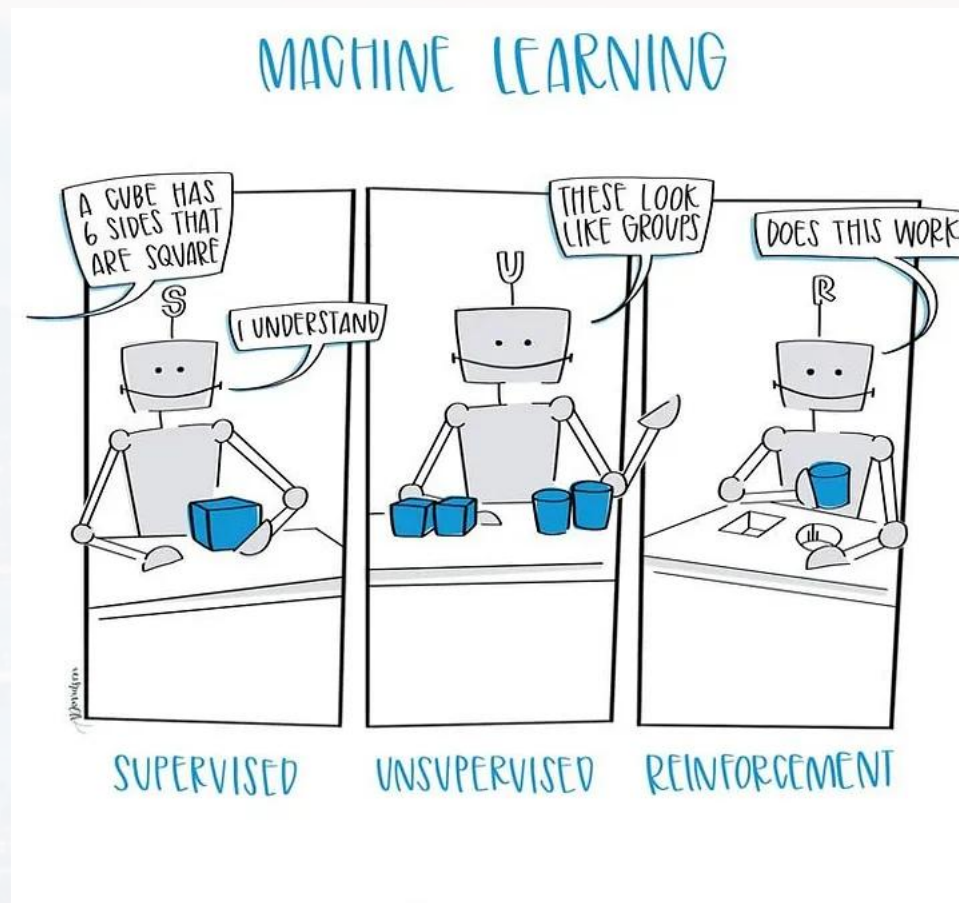
... but maybe there are ways to learn *without* training data





## Outline

- K - means
- GMM
- trees



## Outline

- K - means

- GMM

- trees



idea: finding the barycenter of a cluster and thereby assign the datapoints accordingly

goal: minimizing

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{n,k} \|x_n - \mu_k\|^2$$

$\mu_k$ : barycenter of cluster  $k$

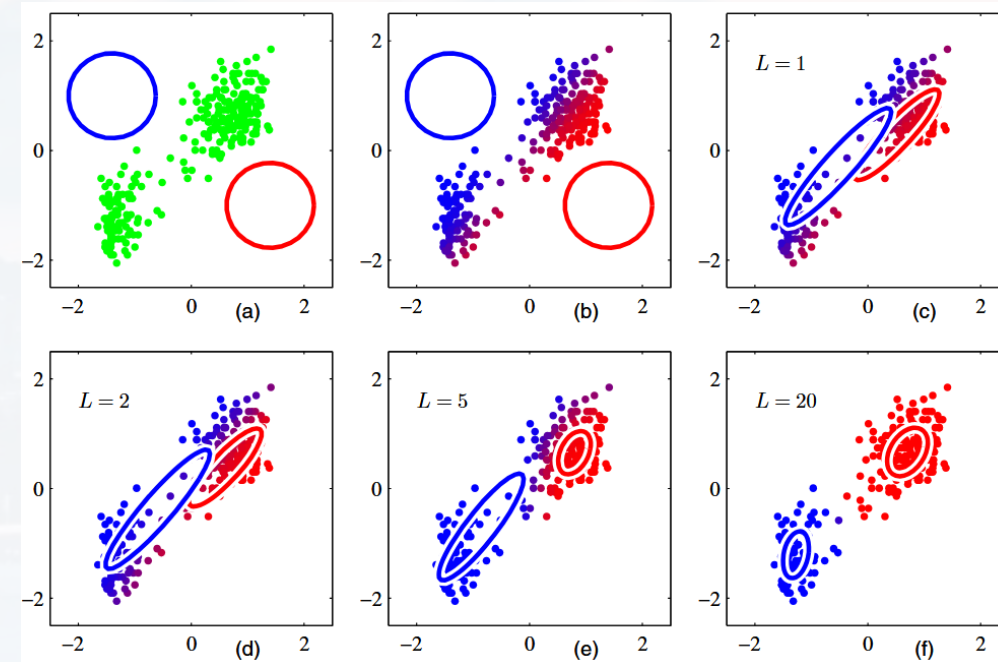
indicator function  $r_{n,k} \in \{0, 1\}$

assigning  $x_n$  to its closed mean

$$r_{n,k} = \begin{cases} 1, & \text{if } k = \operatorname{argmin}_j \|x_n - \mu_j\|^2 \\ 0, & \text{else} \end{cases}$$

$$\frac{\partial J}{\partial \mu_k} = 0 \quad \longrightarrow \quad \mu_k = \frac{\sum_{n=1}^N r_{n,k} x_n}{\sum_{n=1}^N r_{n,k}}$$

|     |                          |
|-----|--------------------------|
| $K$ | : number of cluster      |
| $N$ | : number of observations |



K – means is an iterative process

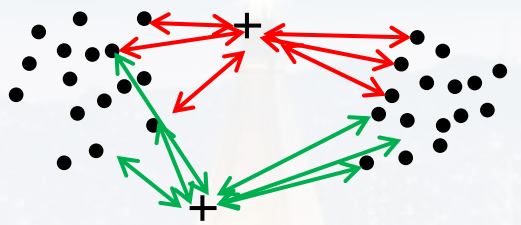


idea:



a) assign  $k$  means randomly

$$r_{n,k} = \begin{cases} 1, & \text{if } k = \operatorname{argmin}_j \|x_n - \mu_j\|^2 \\ 0, & \text{else} \end{cases}$$



b) calculate *distance* from each point to each mean

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{n,k} \|x_n - \mu_k\|^2$$



c) assign each point to its closest mean



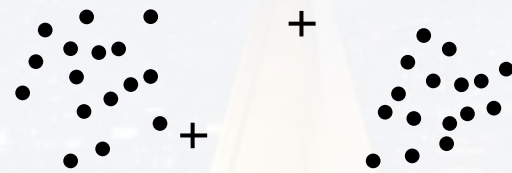
d) update the means accordingly



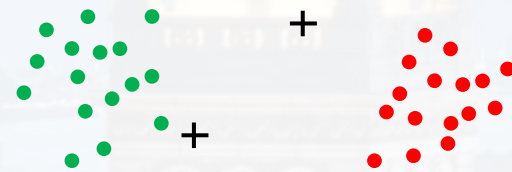
idea:



d) update the means accordingly



e) go back to b)



$$\begin{aligned} K &: \text{number of cluster} \\ N &: \text{number of observations} \\ \mu_k &: \text{barycenter of the cluster} \\ r_{n,k} &= \begin{cases} 1, & \text{if } k = \operatorname{argmin}_j \|x_n - \mu_j\|^2 \\ 0, & \text{else} \end{cases} \end{aligned}$$

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{n,k} \|x_n - \mu_k\|^2$$



problem:  $k = \text{number of cluster}$ , is a hyperparameter. How do I know the correct value for  $k$ ?

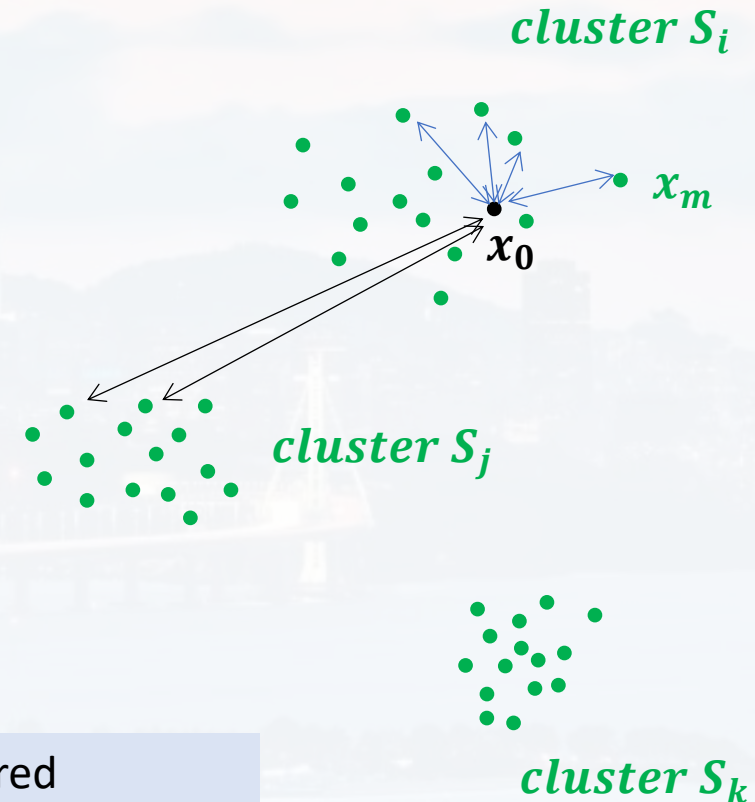
→ silhouette  $\Psi$

- distance  $d_1$  of a data point  $x_0$  to *its assigned cluster*  $S_i$   
vs distance  $d_2$  to *closest cluster (here  $S_j$ )*

$$\Psi(x_0) = \begin{cases} 0 & \text{if } d_1 = 0 \\ \frac{d_2 - d_1}{\max[d_1; d_2]} \end{cases}$$

- average over all points →  $\psi_{tot}$

|    |                                |                         |
|----|--------------------------------|-------------------------|
| if | $\psi_{tot} = 0.75 \dots 1.00$ | → well clustered        |
|    | $\psi_{tot} = 0.50 \dots 0.75$ | → medium clustered      |
|    | $\psi_{tot} = 0.25 \dots 0.50$ | → poorly clustered      |
|    | $\psi_{tot} < 0.25$            | → data has no structure |





problem:  $k$  is a hyperparameter. How do I know the correct value for  $k$ ?

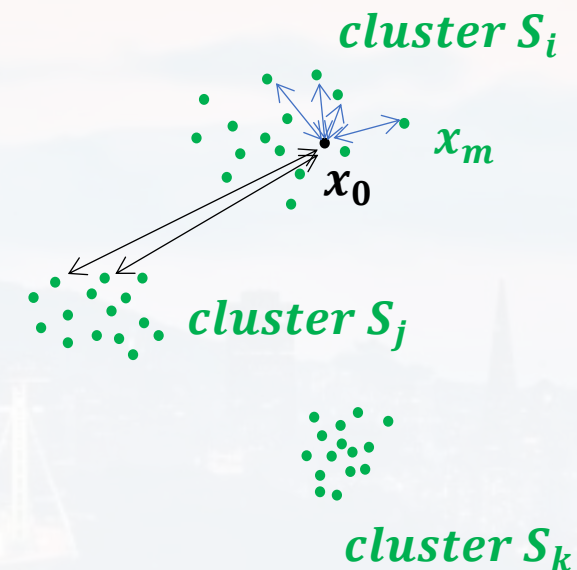
→ silhouette  $\Psi$

- distance  $d_1$  of a data point  $x_0$  to **its assigned cluster**  $S_i$   
vs distance  $d_2$  to **closest cluster (here  $S_j$ )**

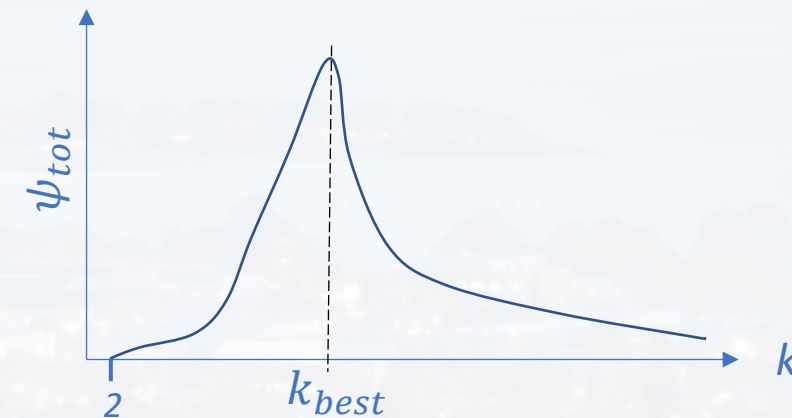
$$\Psi(x_0) = \begin{cases} 0 & \text{if } d_1 = 0 \\ \frac{d_2 - d_1}{\max[d_1; d_2]} \end{cases}$$

- average over all points →  $\psi_{tot}$

$\psi_{tot} = 0.75 \dots 1.00$  → well clustered  
 $\psi_{tot} = 0.50 \dots 0.75$  → medium clustered  
 $\psi_{tot} = 0.25 \dots 0.50$  → poorly clustered  
 $\psi_{tot} < 0.25$  → data has no structure



ideal world →





```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

our standard libraries

for having different  
distances available

```
from pyclustering.utils.metric import *
```

```
from nltk.cluster.kmeans import KMeansClusterer
```

```
from sklearn.metrics import silhouette_samples, silhouette_score
```

```
from sklearn import datasets
```

calling the famous "iris" data set

calculating silhouette  
coefficient for different k

performing k-means



```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from pyclustering.utils.metric import *
from nltk.cluster.kmeans import KMeans(
from sklearn.metrics import silhouette_
from sklearn import datasets
```

```
iris = datasets.load_iris()
```

```
iris.DESCR
```

Iris plants dataset

**\*\*Data Set Characteristics:\*\***

:Number of Instances: 150 (50 in each of three classes)  
:Number of Attributes: 4 numeric, predictive attributes and the class  
:Attribute Information:

- sepal length in cm
- sepal width in cm
- petal length in cm
- petal width in cm
- class:

- Iris-Setosa
- Iris-Versicolour
- Iris-Virginica

ideal world: three distinct cluster

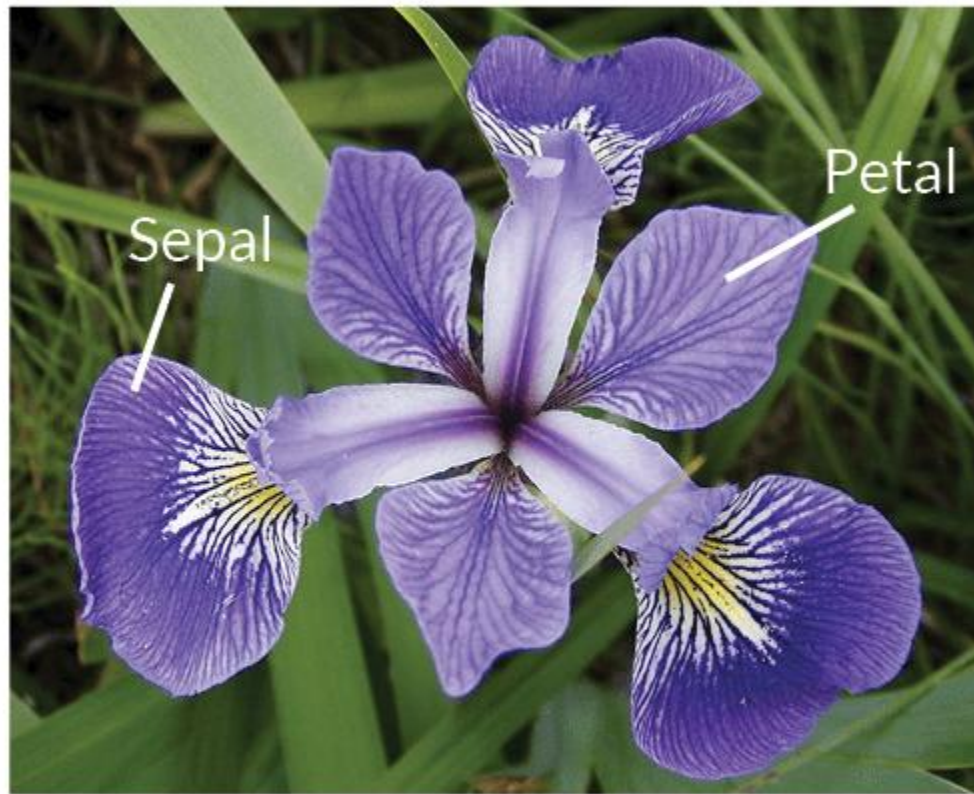
:Summary Statistics:

|               | Min | Max | Mean | SD   | Class Correlation |
|---------------|-----|-----|------|------|-------------------|
| sepal length: | 4.3 | 7.9 | 5.84 | 0.83 | 0.7826            |
| sepal width:  | 2.0 | 4.4 | 3.05 | 0.43 | -0.4194           |
| petal length: | 1.0 | 6.9 | 3.76 | 1.76 | 0.9490 (high!)    |
| petal width:  | 0.1 | 2.5 | 1.20 | 0.76 | 0.9565 (high!)    |



```
iris = datasets.load_iris()
```

```
iris.DESCR
```



**Iris Versicolor**



**Iris Setosa**



**Iris Virginica**



loading & exploring the data:

```
iris = datasets.load_iris()
```

```
iris.DESCR
```

```
iris.feature_names
```

```
iris.target_names
```

```
['sepal length (cm)',  
'sepal width (cm)',  
'petal length (cm)',  
'petal width (cm)']
```

four features → 4D

```
array(['setosa', 'versicolor', 'virginica'])
```

check out the Jupyter Notebook [Walk\\_Through\\_Kmeans.ipynb](#)

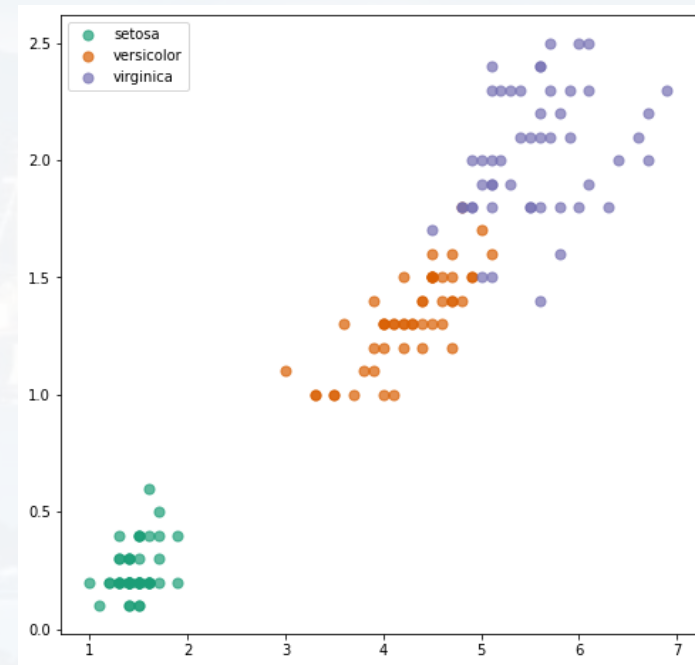
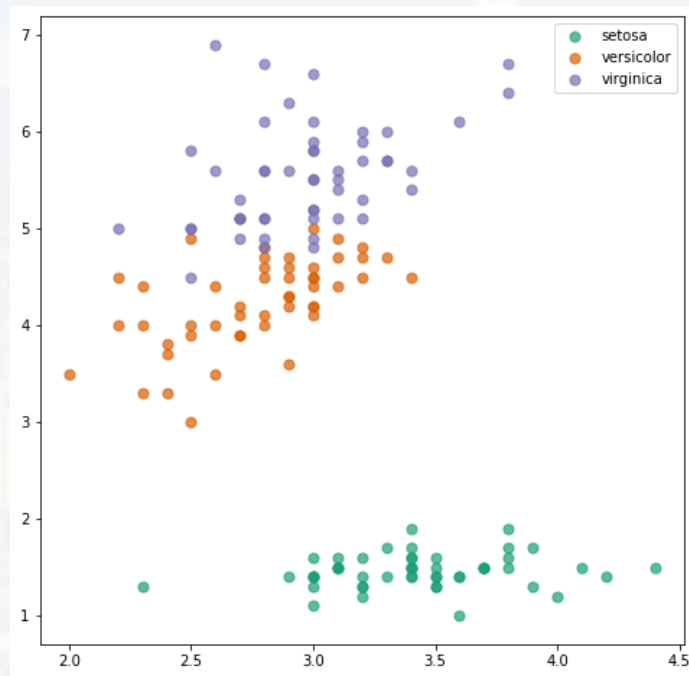
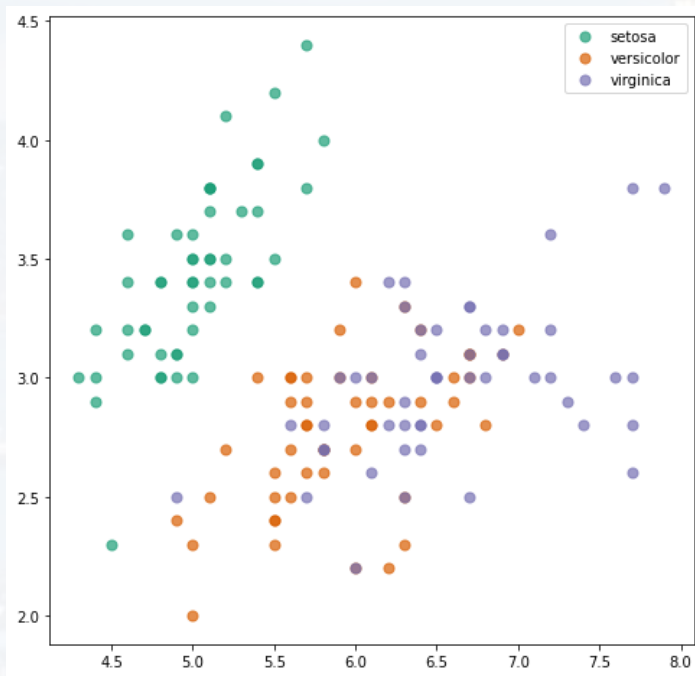
- plotting the data
- running k-means
- evaluating the result



```
['sepal length (cm)',  
'sepal width (cm)',  
'petal length (cm)',  
'petal width (cm)']
```

4D dataset → plotting two components

- **plotting the data**
- running k-means
- evaluating the result





```
nClust    = 3  
rep       = 25  
dist      = distance_metric(type_metric.EUCLIDEAN)
```

```
my_model  = KMeansClusterer(nClust, distance = dist,\n                             repeats   = rep,\n                             avoid_empty_clusters = True)
```

```
PredLabels = my_model.cluster(X2D,\n                               assign_clusters = True)
```

```
Center    = my_model.means()
```

- plotting the data
- **running k-means**
- evaluating the result

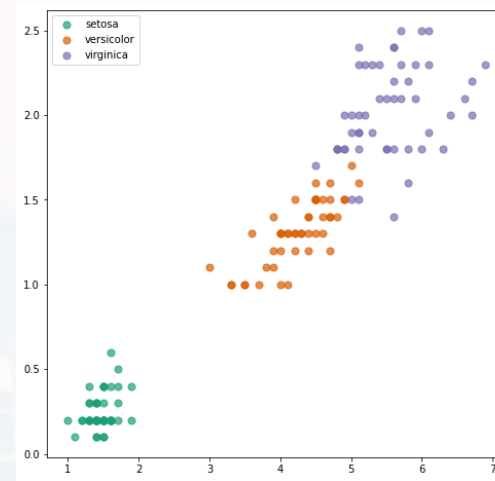
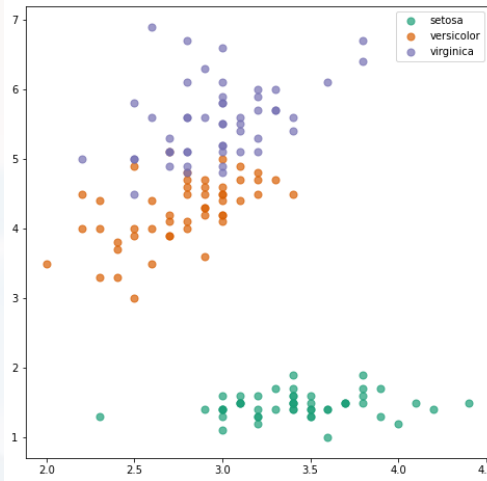
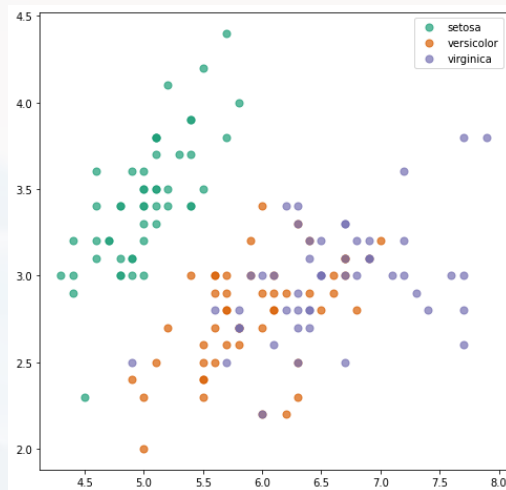
we need to “guess” the number of cluster

the **initial** means are assigned randomly.  
→ repeat the procedure 25 times  
→ avoiding local minimum,

The features are measured in cm, i. e. the correct distance to pick here is Euclidean

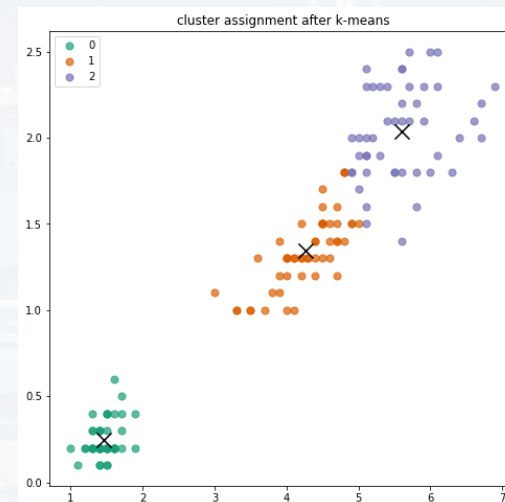
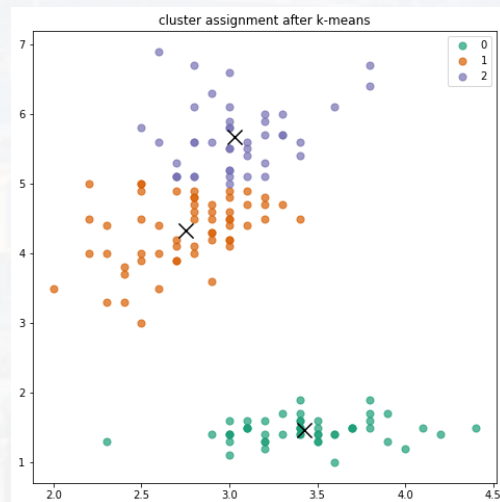
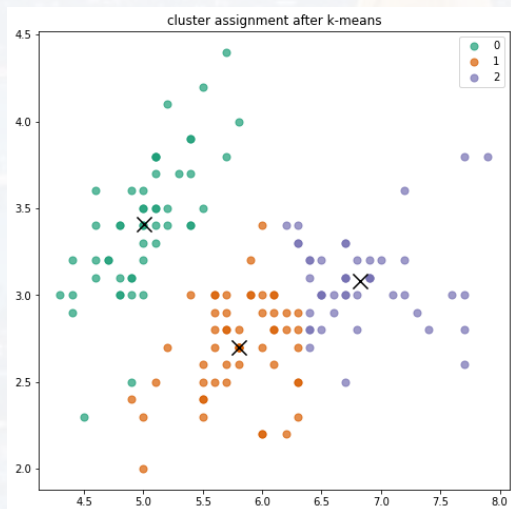


true classes



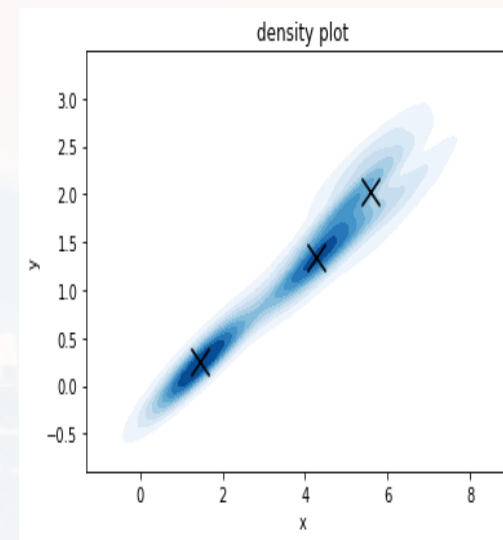
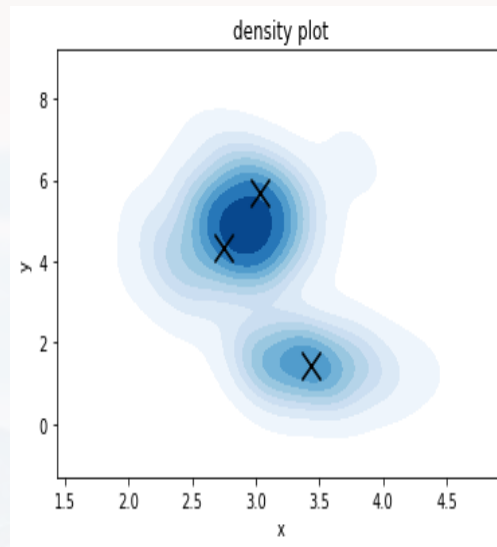
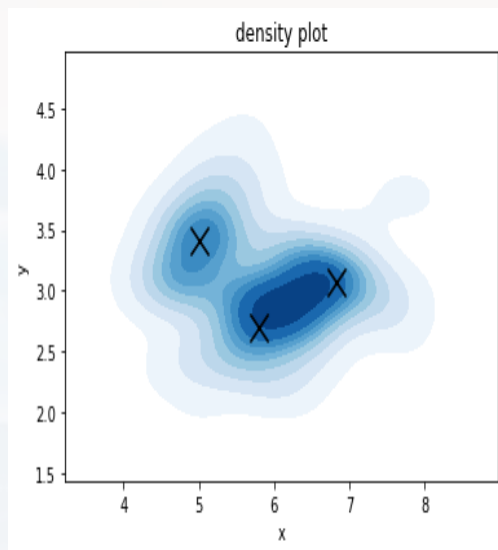
- plotting the data
- **running k-means**
- evaluating the result

assigned classes



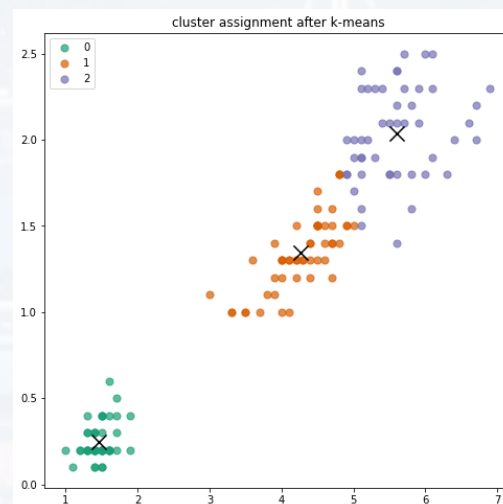
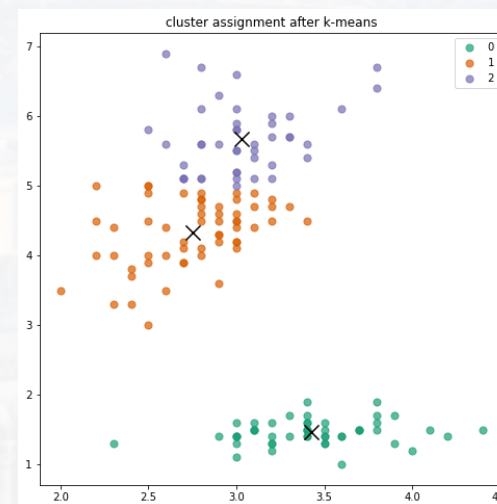
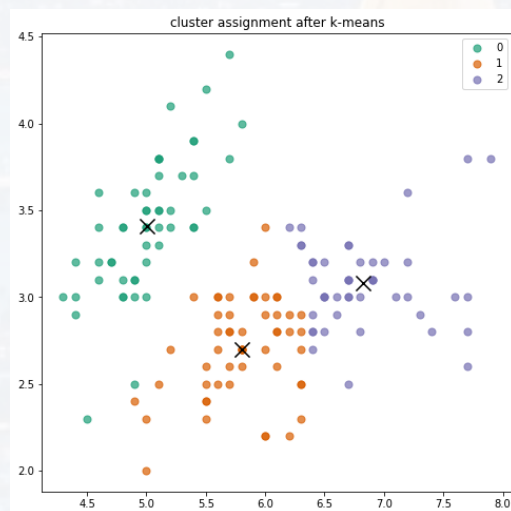


density



- plotting the data
- **running k-means**
- evaluating the result

assigned classes

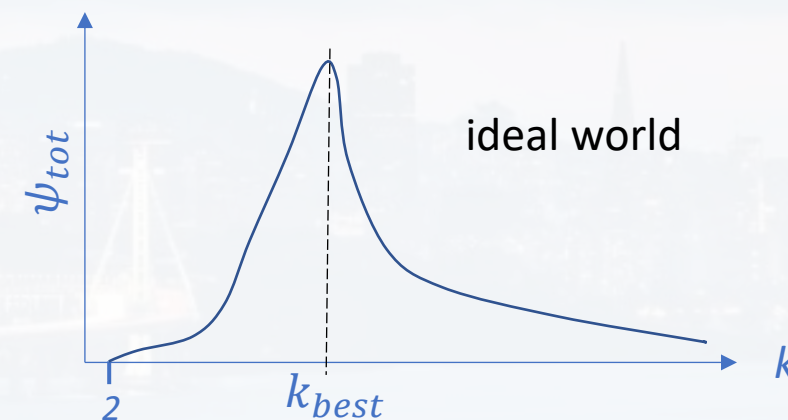
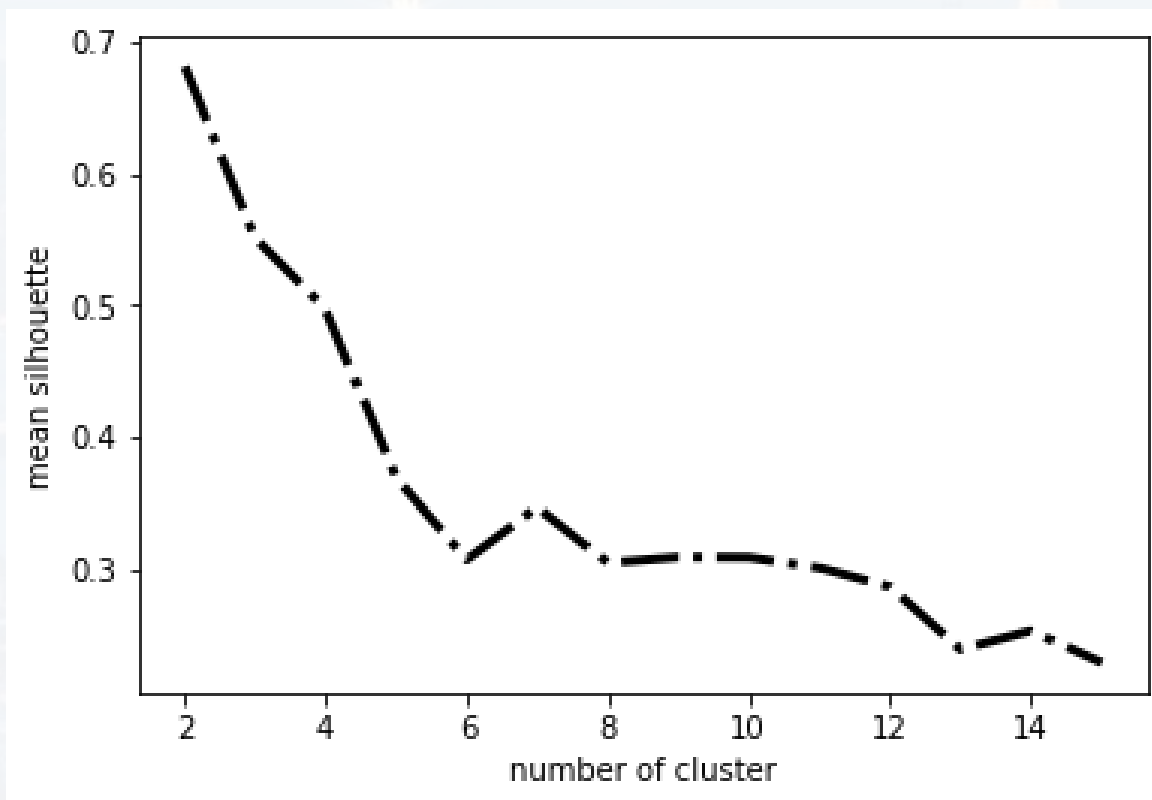




we run k-means now for the **full 4D** dataset  
+ evaluate clustering with silhouette

- plotting the data
- running k-means
- **evaluating the result**

`silhouette_score(X, Labels)`

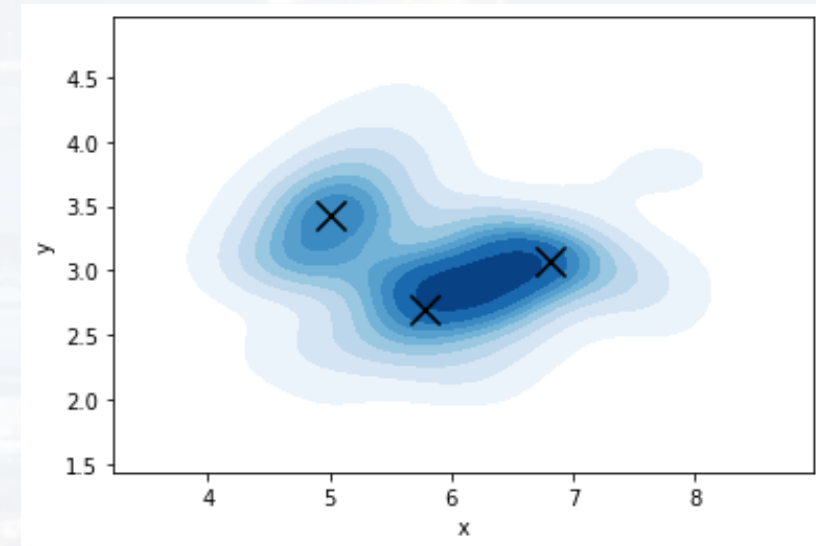
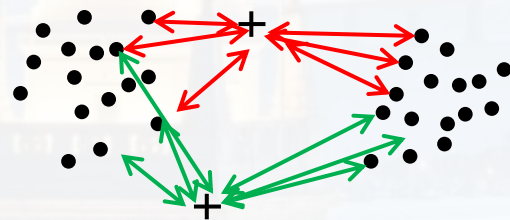


**accuracy for 4D  $k = 3$ : 90%**



### summary:

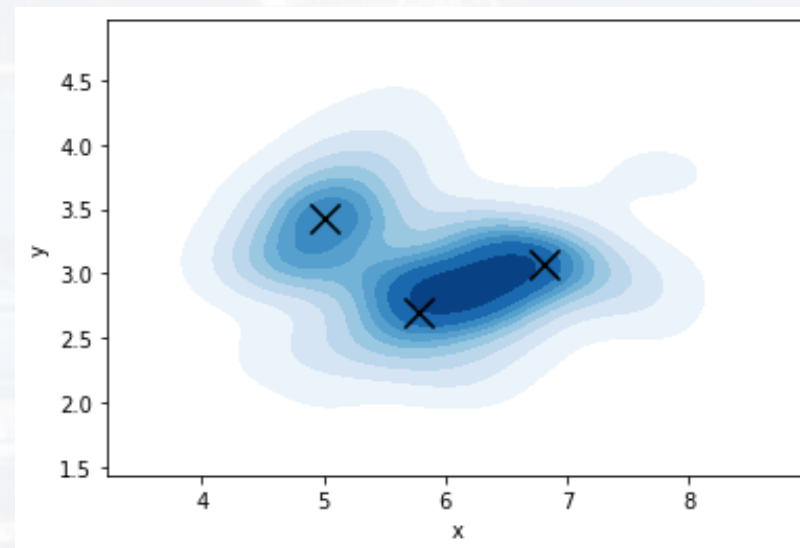
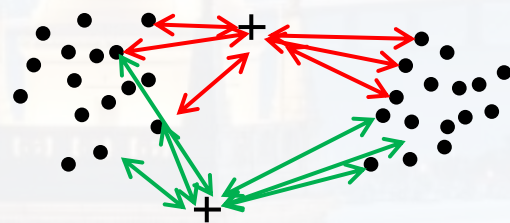
- simple and fast
- unsupervised
- $k$  has to be given  $\rightarrow$  silhouette for determining best  $k$
- problems if cluster have unusual shapes  
(elongated, hollow inside, scattered)
- **image segmentation!**

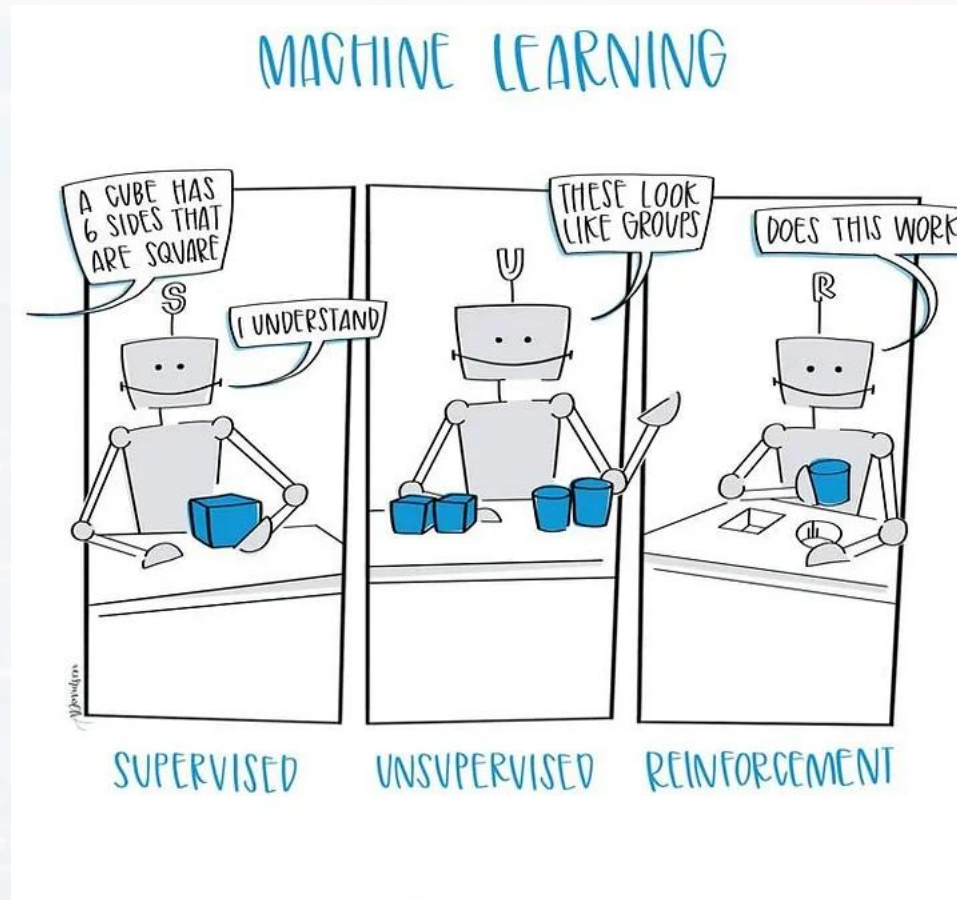




topics for the discussion/office hour:

- What is a *distance*?
- Which are different distances?
- When to use which distance?





## Outline

- K - means

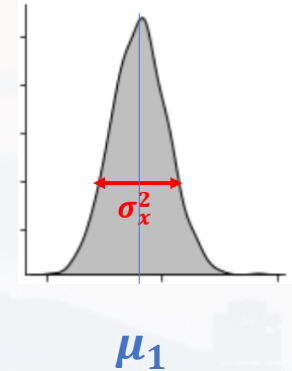
- **GMM**

- trees

## Gaussian Mixture Models

one feature

$$N_1(x_1) = \frac{1}{\sqrt{2\pi \sigma_{x1}^2}} \exp \left\{ -\frac{1}{2} \left( \frac{x_1 - \mu_1}{\sigma_{x1}} \right)^2 \right\}$$

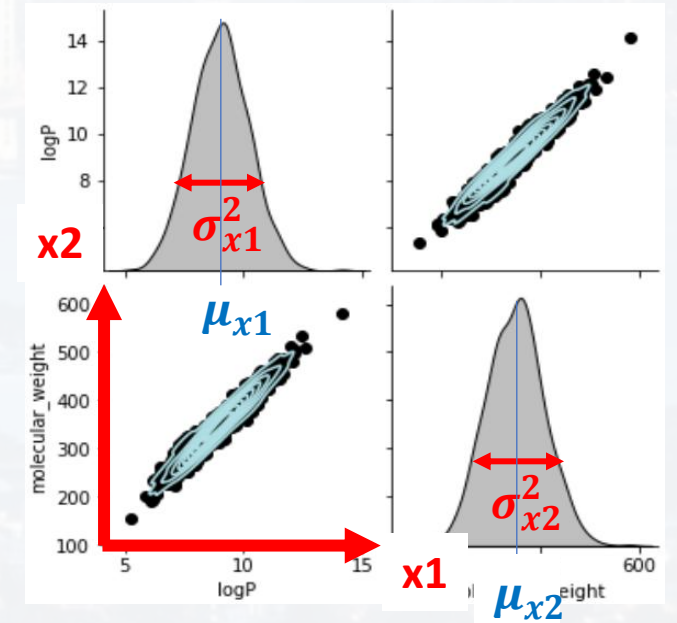


two features

$$\Sigma = \begin{pmatrix} \sigma_{x1}^2 & cov(x_1, x_2) \\ cov(x_2, x_1) & \sigma_{x2}^2 \end{pmatrix} \quad \text{covariance matrix}$$

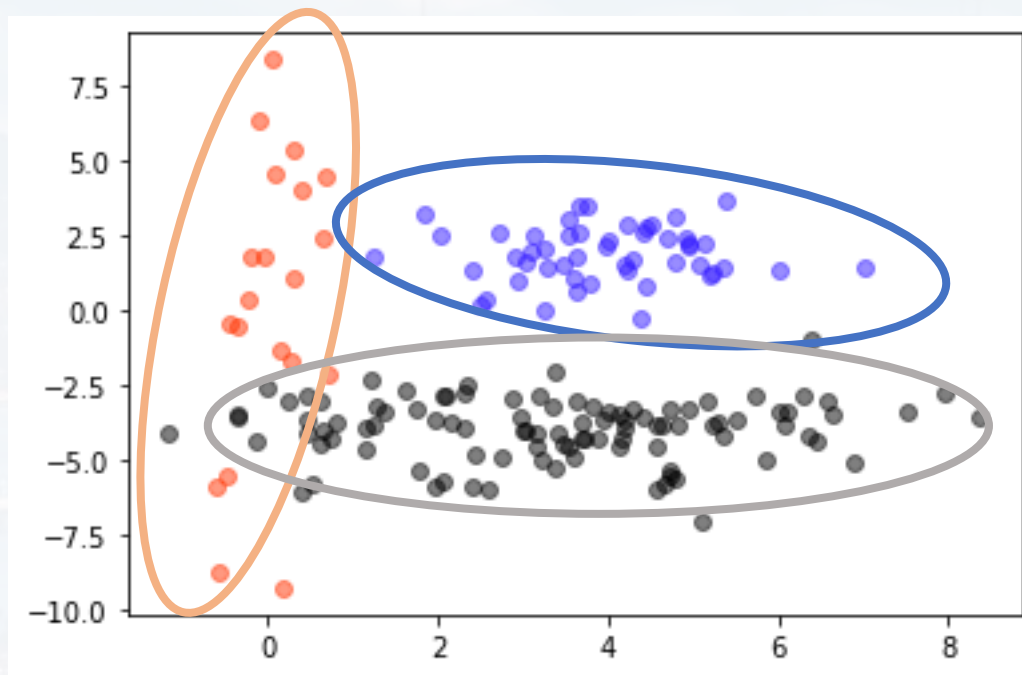
$$\begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix}^T \Sigma^{-1} \begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix} \quad \text{see PCA lecture}$$

$$N_2(x_1, x_2) = \frac{1}{2\pi \det(\Sigma)^{1/2}} \exp \left\{ -\frac{1}{2} \left[ \begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix}^T \Sigma^{-1} \begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix} \right] \right\}$$

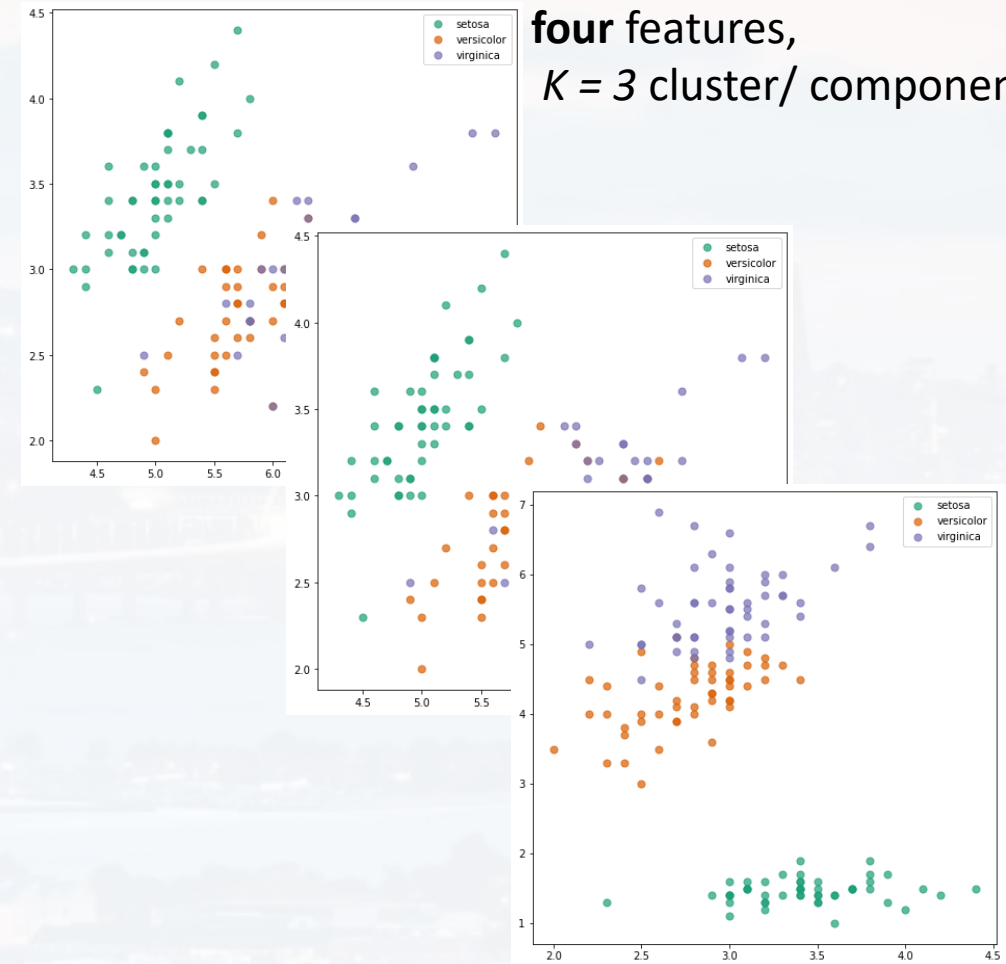


## Gaussian Mixture Models

two features,  $K = 3$  cluster/components



four features,  
 $K = 3$  cluster/ components

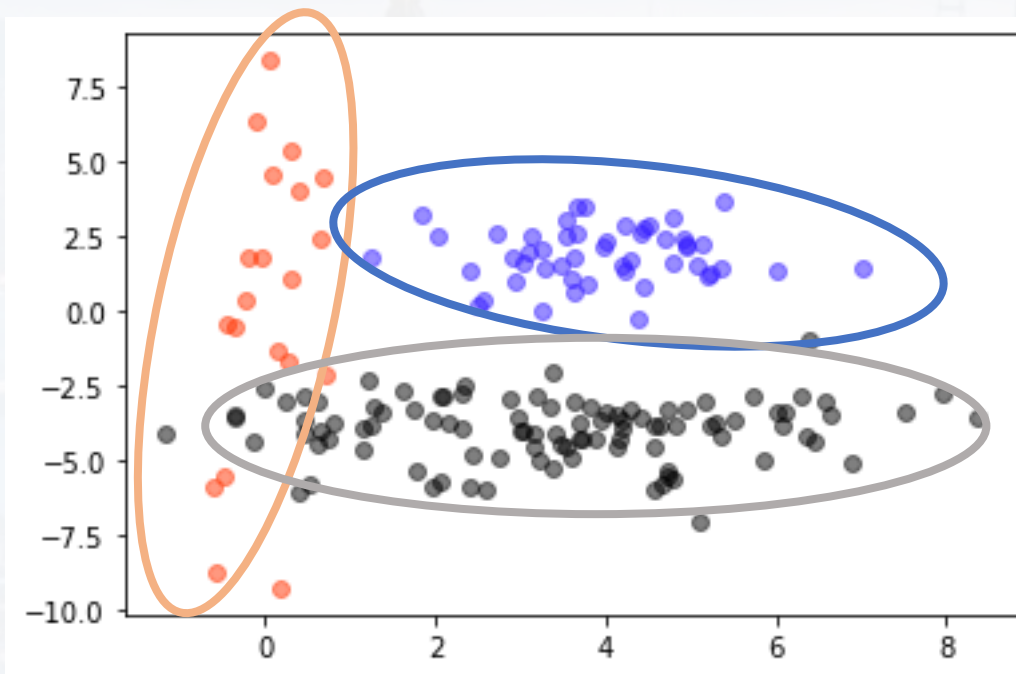


## Gaussian Mixture Models

|                 |                      |
|-----------------|----------------------|
| $K$             | : number of cluster  |
| $\pi_k$         | : mixing coefficient |
| $\mathcal{C}_k$ | : cluster $k$        |

f features

$$\mathcal{N}_k(x|\mu_k, \Sigma_k) = \frac{1}{(2\pi)^{f/2} \det(\Sigma_k)^{1/2}} \exp \left[ -\frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) \right]$$



two features,  $K = 3$  cluster/components

$$\begin{aligned}
 P(x) &= \sum_{k=1}^K P(x|\mathcal{C}_k) P(\mathcal{C}_k) \\
 &= \sum_{k=1}^K \mathcal{N}_k(x|\mu_k, \Sigma_k) \pi_k
 \end{aligned}$$

probability to pick a cluster  $\mathcal{C}_k$

## Gaussian Mixture Models

|         |                          |
|---------|--------------------------|
| $N$     | : number of observations |
| $K$     | : number of cluster      |
| $\pi_k$ | : mixing coefficient     |

$$P(x) = \sum_{k=1}^K P(x|\mathbf{C}_k) P(\mathbf{C}_k) = \sum_{k=1}^K \mathcal{N}_k(x|\mu_k, \Sigma_k) \pi_k \quad \text{marginalization}$$

indicator variable  $z_k \in \{0, 1\}$

goal:  $P(z_k = 1|x) = \frac{P(z_k = 1) P(x|z_k = 1)}{P(x)} = \frac{\pi_k \mathcal{N}_k(x|\mu_k, \Sigma_k)}{\sum_{j=1}^K \mathcal{N}_j(x|\mu_j, \Sigma_j) \pi_j}$

Bayes Theorem

via maximizing likelihood by finding best  $\theta$   $L = \ln[P(x|\pi, \mu, \Sigma)] = \ln \left[ \prod_{n=1}^N \left\{ \sum_{k=1}^K \pi_k \mathcal{N}_k(x_n|\mu_k, \Sigma_k) \right\} \right]$

model parameter  $\theta = \{\pi, \mu, \Sigma\}$   $= \sum_{n=1}^N \ln \left\{ \sum_{k=1}^K \pi_k \mathcal{N}_k(x_n|\mu_k, \Sigma_k) \right\}$

## Gaussian Mixture Models Expectation Maximization

|         |                          |
|---------|--------------------------|
| $N$     | : number of observations |
| $K$     | : number of cluster      |
| $\pi_k$ | : mixing coefficient     |

1) initialize  $\theta = \{\pi, \mu, \Sigma\}$  and  $P(z_k = 1|x_n)$

2) Expectation step  $t$ :

$$P(z_k = 1|x_n) = \frac{\pi_k \mathcal{N}_k(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^K \mathcal{N}_j(x_n|\mu_j, \Sigma_j) \pi_j} \quad \text{i.e. evaluate } P(z|x, \theta)$$

3) Maximization step  $t$  (for example MLE):

$$\frac{\partial L}{\partial \mu} = 0 \quad \mu_k^t = \frac{1}{N_k} \sum_{n=1}^N P(z_k = 1|x_n) x_n$$

$$N_k^t = \sum_{n=1}^N P(z_k = 1|x_n)$$

$$\frac{\partial L}{\partial \Sigma} = 0 \quad \Sigma_k^t = \frac{1}{N_k} \sum_{n=1}^N P(z_k = 1|x_n) (x_n - \mu_k^t) (x_n - \mu_k^t)^T$$

$$\frac{\partial L}{\partial \pi} = 0 \quad \pi_k^t = \frac{N_k}{N}$$

## Gaussian Mixture Models

idea: fitting the data to a GMM → analytical functions to **calculate** probabilities for labels

different algorithms:

- Bayesian
- **Expectation Maximization**
- ...

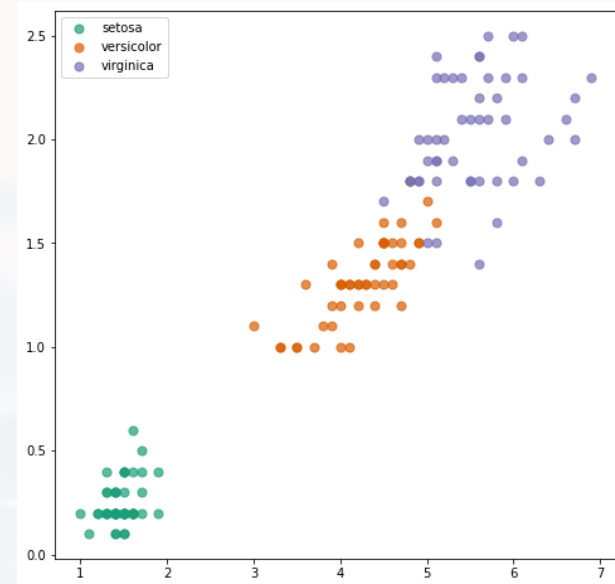
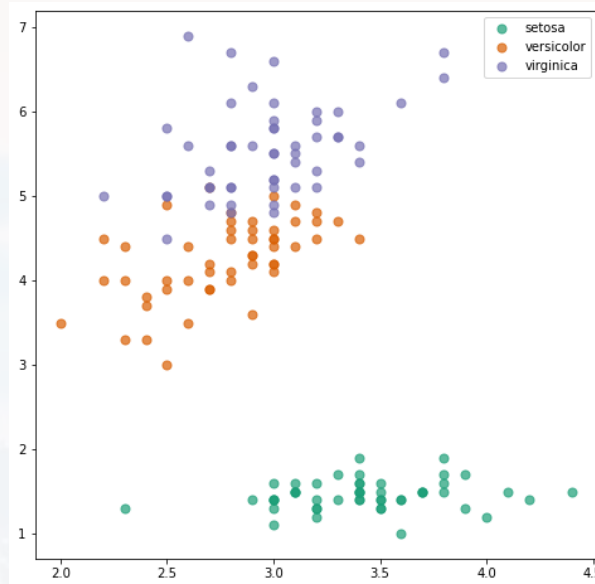
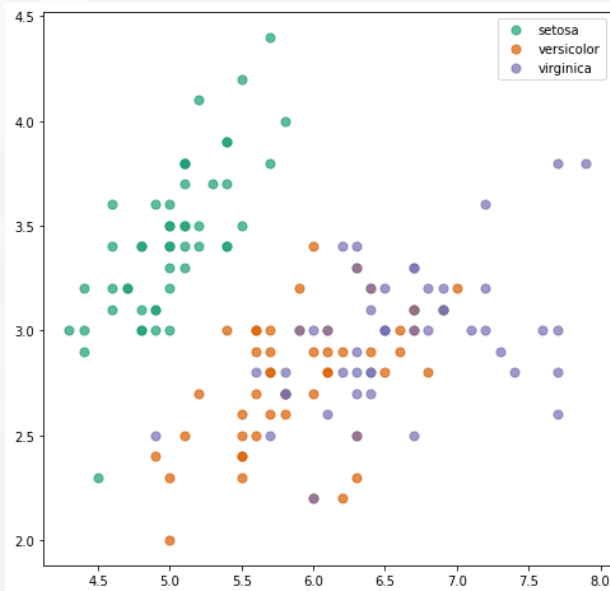
```
my_model = GaussianMixture(n_components = k, random_state = 0).fit(X)
```

```
Center = my_model.means_  
PredLabels = my_model.predict(X)
```

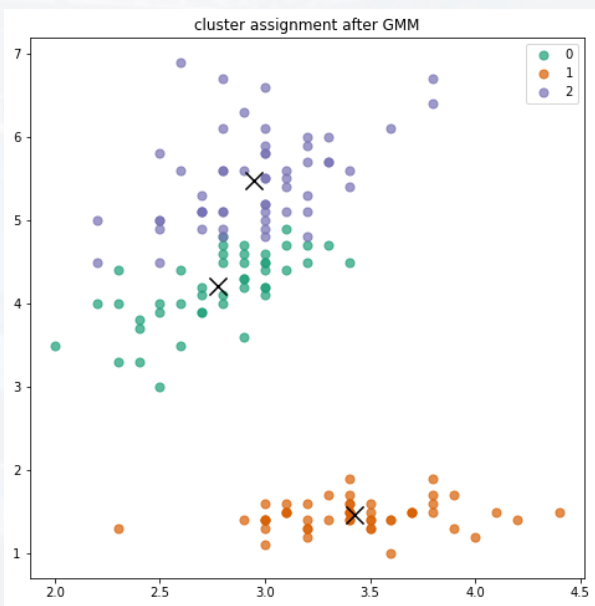
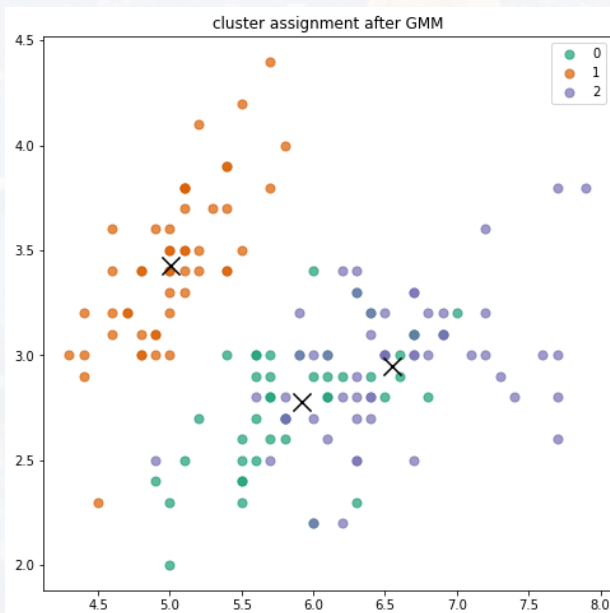
check out [Walk\\_Through\\_GMM.ipynb](#)

setting initial labels  
randomly

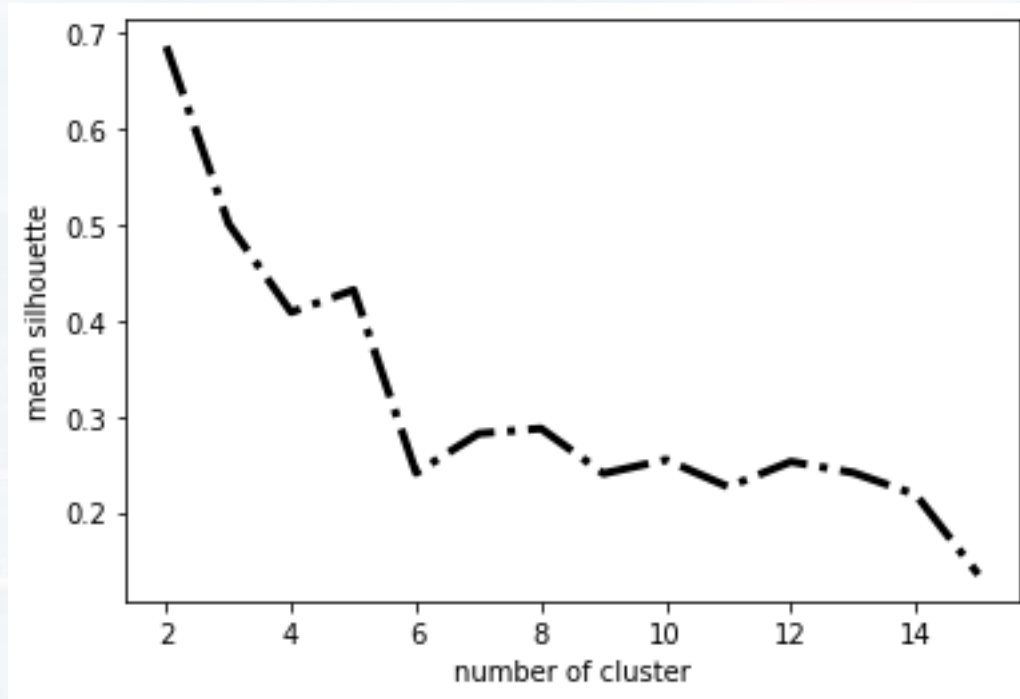
true classes



assigned classes



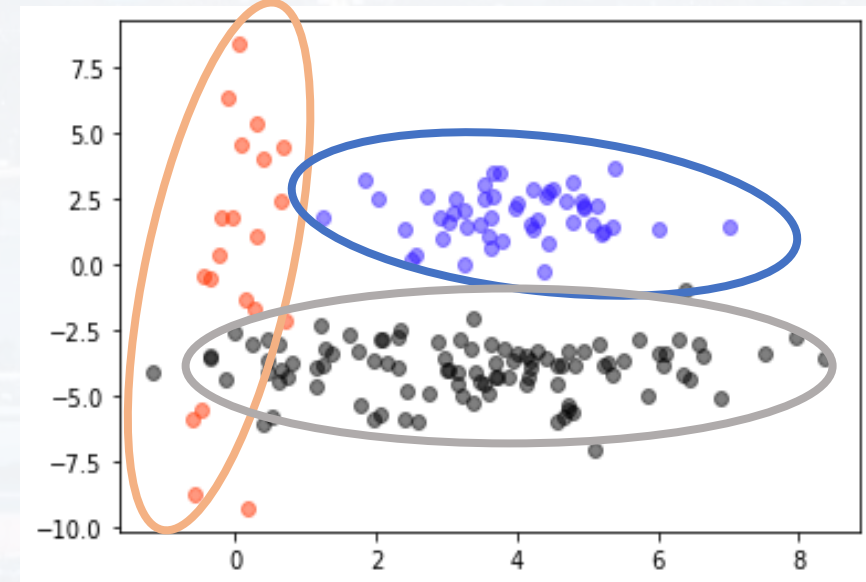
we run the GMM now for the **full 4D** dataset  
+ evaluate clustering with silhouette

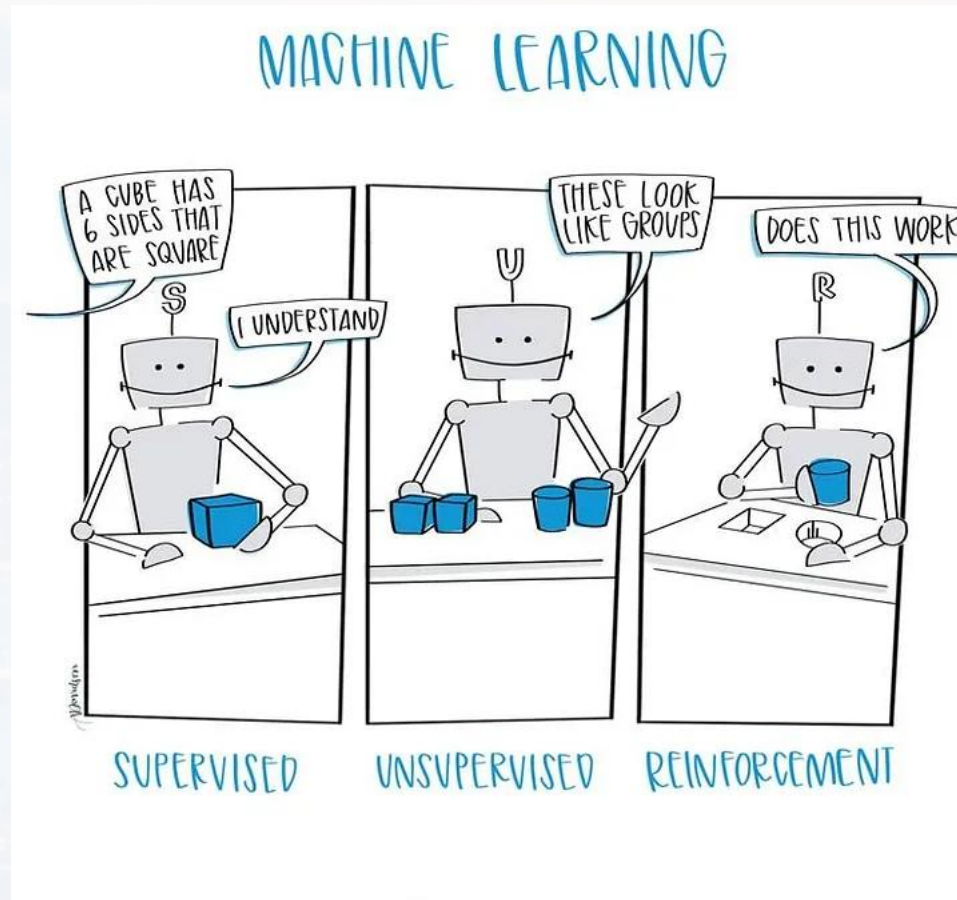


accuracy for 4D  $k = 3$ : 93%

topics for the discussion/office hour:

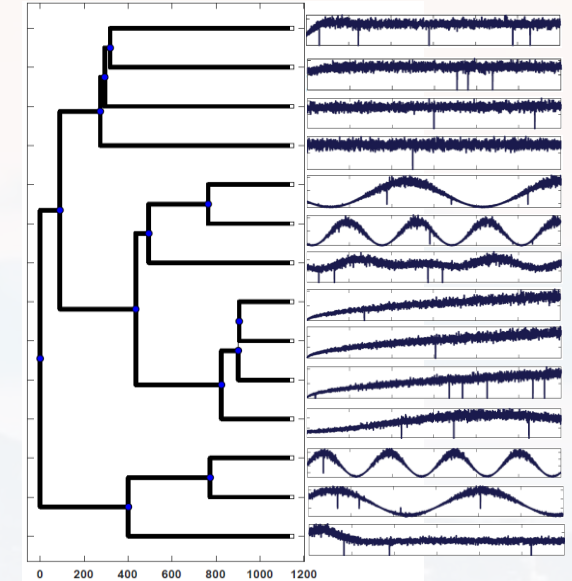
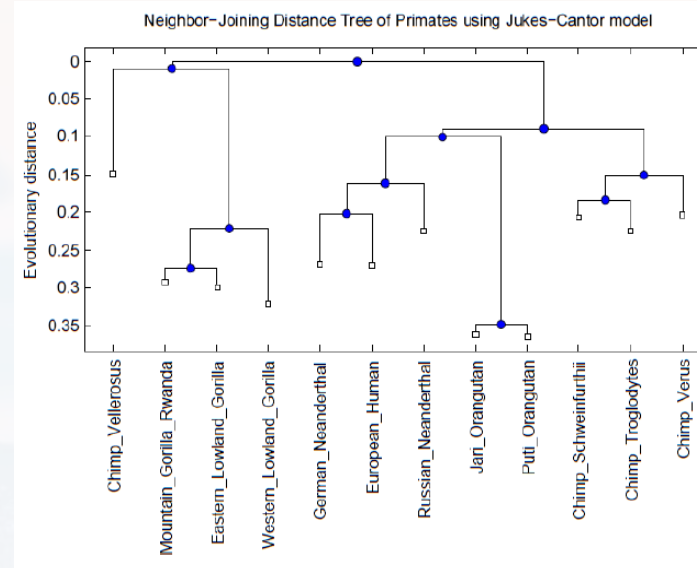
- EM algorithm
- mean, variance and covariance in more detail



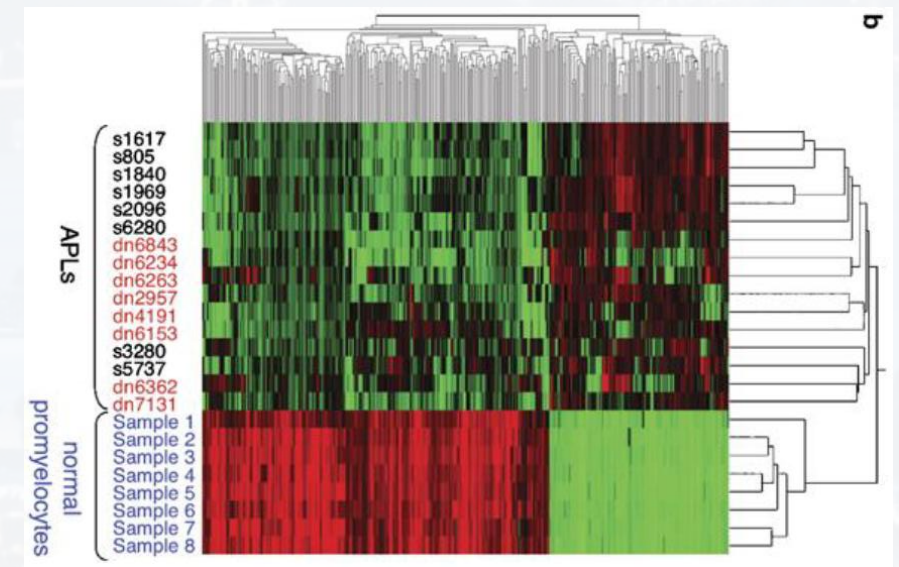


## Outline

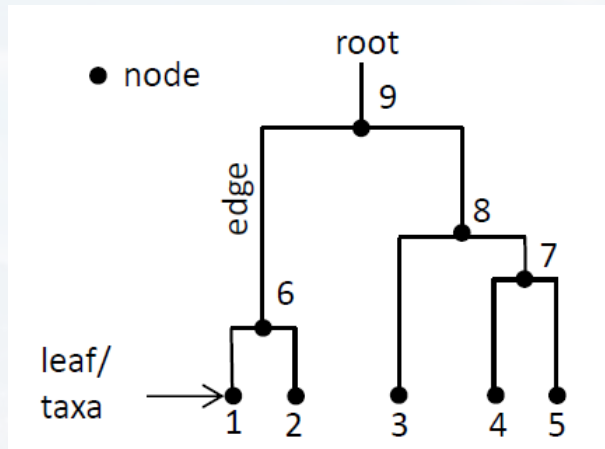
- K - means
- GMM
- **trees**



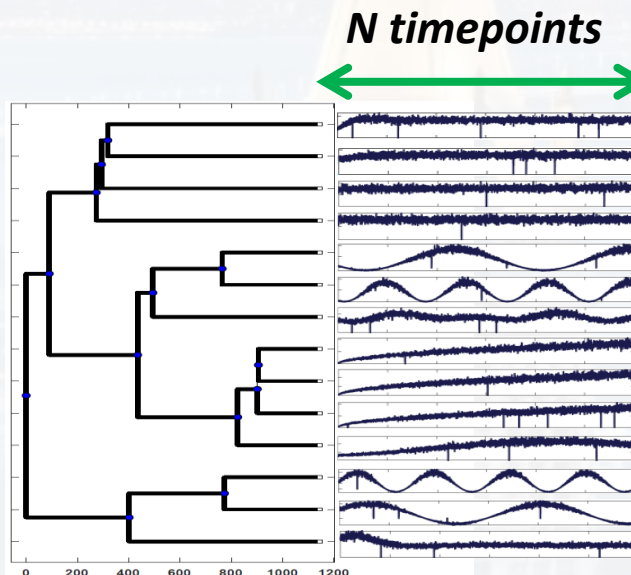
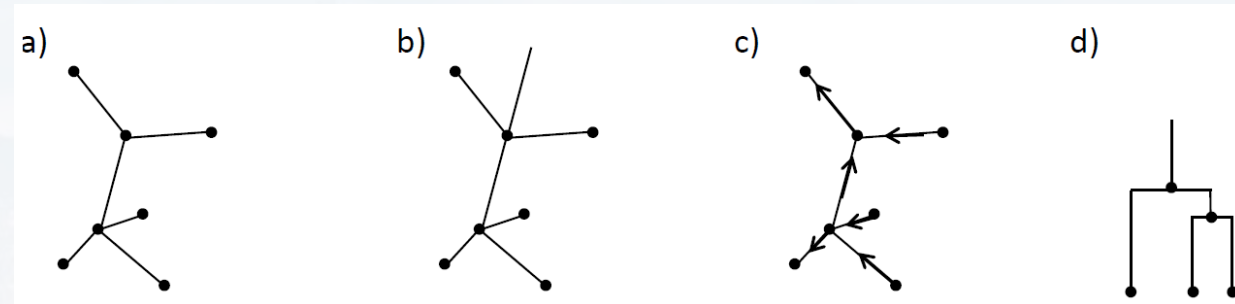
- What is a tree?
- Different kinds of trees...?
- How to build a tree?
- Why do we need trees?
- Examples...



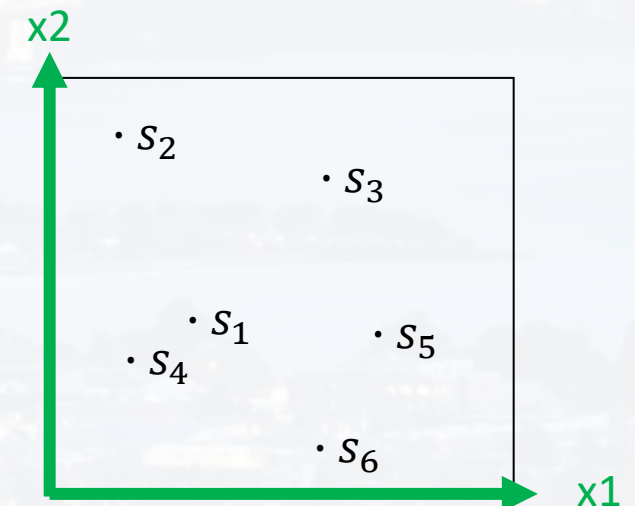
trees are a subclass of graphs (but: not fully connected  $\rightarrow$  “hierarchy”, no loops):



- a) unrooted, undirected multinary tree
- b) rooted, undirected multinary tree
- c) unrooted, directed multinary tree
- d) rooted, undirected binary tree

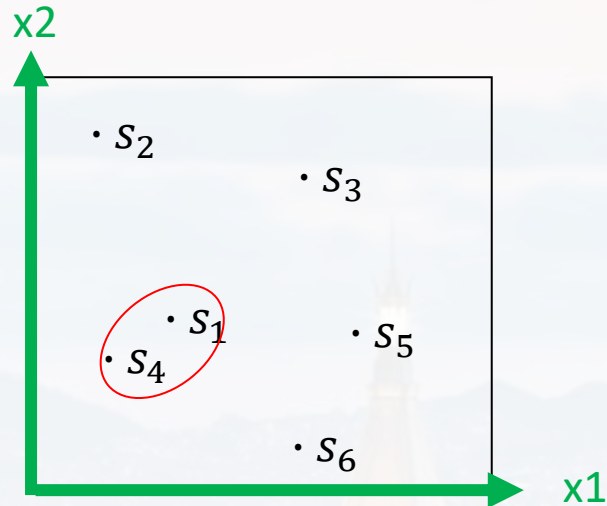


*each sample  $s$  is  
a vector of  $N$  rows,  
hence, a data point  
in  $N$ -D*



constructing trees:

- calculating a distance between each pair of samples

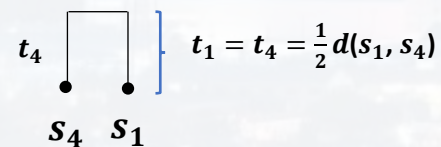


|       | $s_1$ | $s_2$         | $s_3$         | $s_4$         | $s_5$         | $s_6$         |
|-------|-------|---------------|---------------|---------------|---------------|---------------|
| $s_1$ | 0     | $d(s_1, s_2)$ | $d(s_1, s_3)$ | $d(s_1, s_4)$ | $d(s_1, s_5)$ | $d(s_1, s_6)$ |
| $s_2$ |       | 0             | $d(s_2, s_3)$ | $d(s_2, s_4)$ | $d(s_2, s_5)$ | $d(s_2, s_6)$ |
| $s_3$ |       |               | 0             | $d(s_3, s_4)$ | $d(s_3, s_5)$ | $d(s_3, s_6)$ |
| $s_4$ |       |               |               | 0             | $d(s_4, s_5)$ | $d(s_4, s_6)$ |
| $s_5$ |       |               |               |               | 0             | $d(s_5, s_6)$ |
| $s_6$ |       |               |               |               |               | 0             |

**Question: What could be a proper distance definition here?**

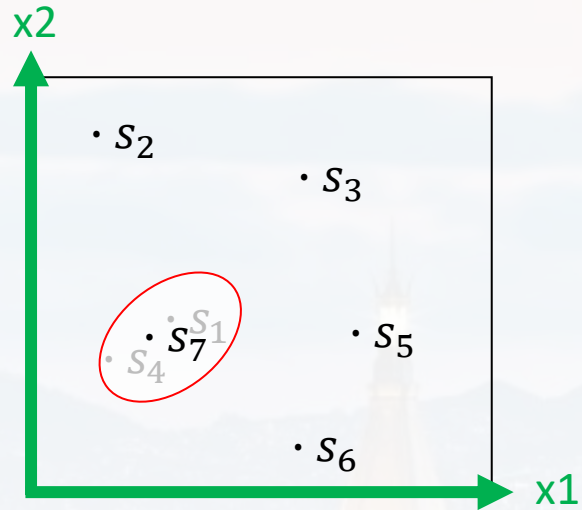
...once a distance has been defined...

→ find the closest pair



$$t_1 = t_4 = \frac{1}{2} d(s_1, s_4)$$

constructing trees:



- calculating a distance between each pair of samples

|       | $s_1$ | $s_2$         | $s_3$         | $s_4$         | $s_5$         | $s_6$         |
|-------|-------|---------------|---------------|---------------|---------------|---------------|
| $s_1$ | 0     | $d(s_1, s_2)$ | $d(s_1, s_3)$ | $d(s_1, s_4)$ | $d(s_1, s_5)$ | $d(s_1, s_6)$ |
| $s_2$ |       | 0             | $d(s_2, s_3)$ | $d(s_2, s_4)$ | $d(s_2, s_5)$ | $d(s_2, s_6)$ |
| $s_3$ |       |               | 0             | $d(s_3, s_4)$ | $d(s_3, s_5)$ | $d(s_3, s_6)$ |
| $s_4$ |       |               |               | 0             | $d(s_4, s_5)$ | $d(s_4, s_6)$ |
| $s_5$ |       |               |               |               | 0             | $d(s_5, s_6)$ |
| $s_6$ |       |               |               |               |               | 0             |

- treat it as a new cluster  $s_{1,4}$
- use average of distance from cluster elements

$$\begin{array}{c}
 \bullet \\
 \boxed{\phantom{S7}} \\
 \bullet \quad \bullet \\
 s_4 \quad s_1
 \end{array}
 \quad
 t_1 = t_4 = \frac{1}{2} d(s_1, s_4)$$



- calculating a distance between each pair of samples

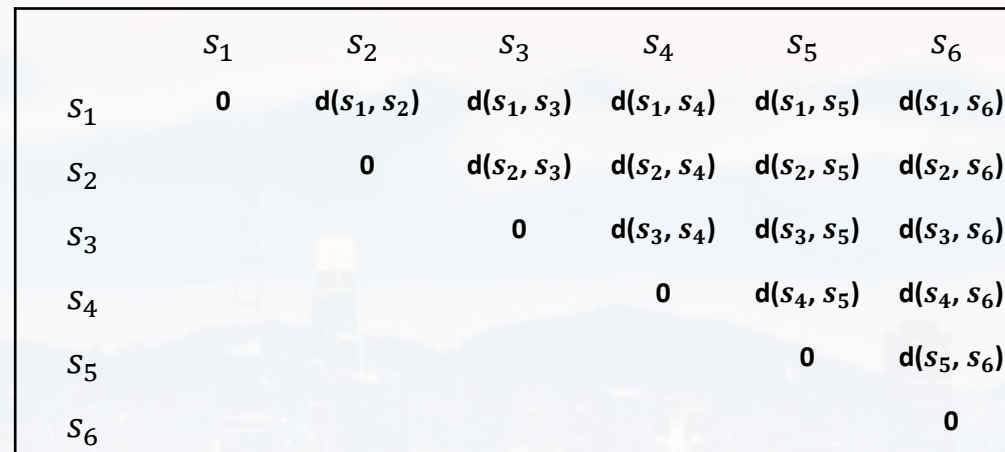
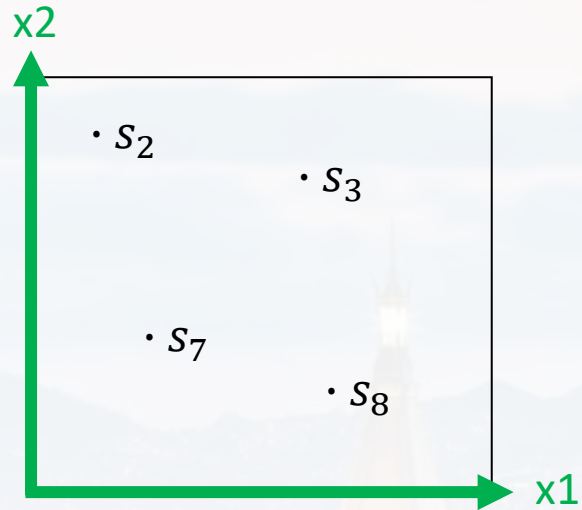


Figure 1 consists of two diagrams. The left diagram shows a path with three vertices labeled  $s_4$ ,  $s_7$ , and  $s_1$  from left to right. A horizontal line connects  $s_4$  and  $s_1$ , with a vertex  $s_7$  on it. A vertical line segment connects  $s_4$  to  $s_7$ , labeled  $t_4$ . Another vertical line segment connects  $s_7$  to  $s_1$ , labeled  $t_1 = t_4 = \frac{1}{2}d(s_1, s_4)$ . A blue bracket on the right side of the diagram groups the two vertical segments. The right diagram shows a path with three vertices labeled  $s_5$ ,  $s_8$ , and  $s_6$  from left to right. A horizontal line connects  $s_5$  and  $s_6$ , with a vertex  $s_8$  on it. A vertical line segment connects  $s_5$  to  $s_8$ , labeled  $t_5$ . Another vertical line segment connects  $s_8$  to  $s_6$ , labeled  $t_6 = t_5 = \frac{1}{2}d(s_5, s_6)$ . A blue bracket on the right side of the diagram groups the two vertical segments.

constructing trees:

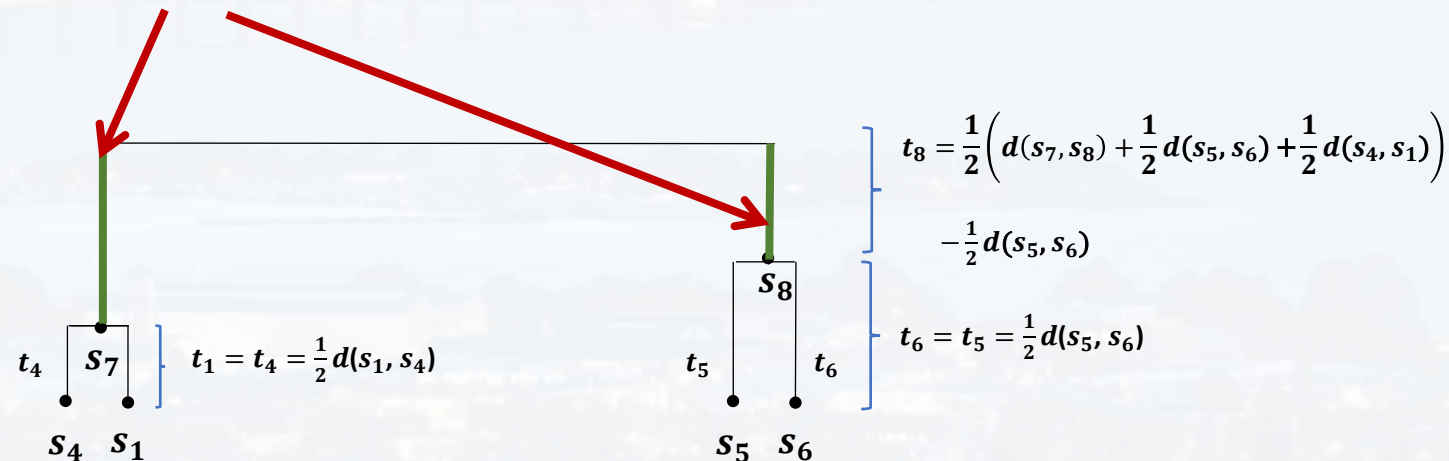


- calculating a distance between each pair of samples

|       | $s_1$ | $s_2$         | $s_3$         | $s_4$         | $s_5$         | $s_6$         |
|-------|-------|---------------|---------------|---------------|---------------|---------------|
| $s_1$ | 0     | $d(s_1, s_2)$ | $d(s_1, s_3)$ | $d(s_1, s_4)$ | $d(s_1, s_5)$ | $d(s_1, s_6)$ |
| $s_2$ |       | 0             | $d(s_2, s_3)$ | $d(s_2, s_4)$ | $d(s_2, s_5)$ | $d(s_2, s_6)$ |
| $s_3$ |       |               | 0             | $d(s_3, s_4)$ | $d(s_3, s_5)$ | $d(s_3, s_6)$ |
| $s_4$ |       |               |               | 0             | $d(s_4, s_5)$ | $d(s_4, s_6)$ |
| $s_5$ |       |               |               |               | 0             | $d(s_5, s_6)$ |
| $s_6$ |       |               |               |               |               | 0             |

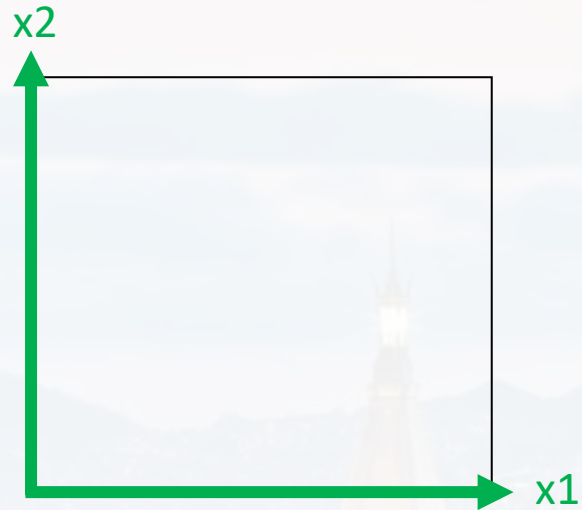
→ and so on....

$$d(s_7, s_8) = \frac{d(s_5, s_7) + d(s_6, s_7)}{2} = \frac{d(s_1, s_5) + d(s_4, s_5) + d(s_1, s_6) + d(s_4, s_6)}{4}$$



constructing trees:

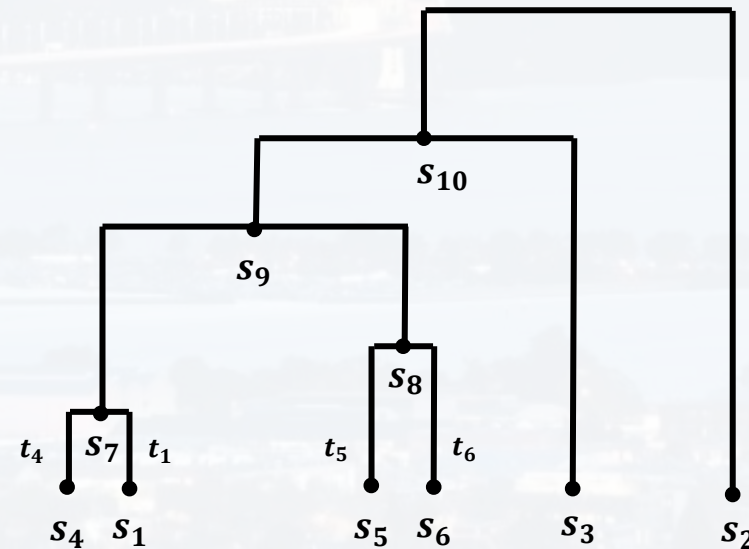
- calculating a distance between each pair of samples



|       | $s_1$ | $s_2$         | $s_3$         | $s_4$         | $s_5$         | $s_6$         |
|-------|-------|---------------|---------------|---------------|---------------|---------------|
| $s_1$ | 0     | $d(s_1, s_2)$ | $d(s_1, s_3)$ | $d(s_1, s_4)$ | $d(s_1, s_5)$ | $d(s_1, s_6)$ |
| $s_2$ |       | 0             | $d(s_2, s_3)$ | $d(s_2, s_4)$ | $d(s_2, s_5)$ | $d(s_2, s_6)$ |
| $s_3$ |       |               | 0             | $d(s_3, s_4)$ | $d(s_3, s_5)$ | $d(s_3, s_6)$ |
| $s_4$ |       |               |               | 0             | $d(s_4, s_5)$ | $d(s_4, s_6)$ |
| $s_5$ |       |               |               |               | 0             | $d(s_5, s_6)$ |
| $s_6$ |       |               |               |               |               | 0             |

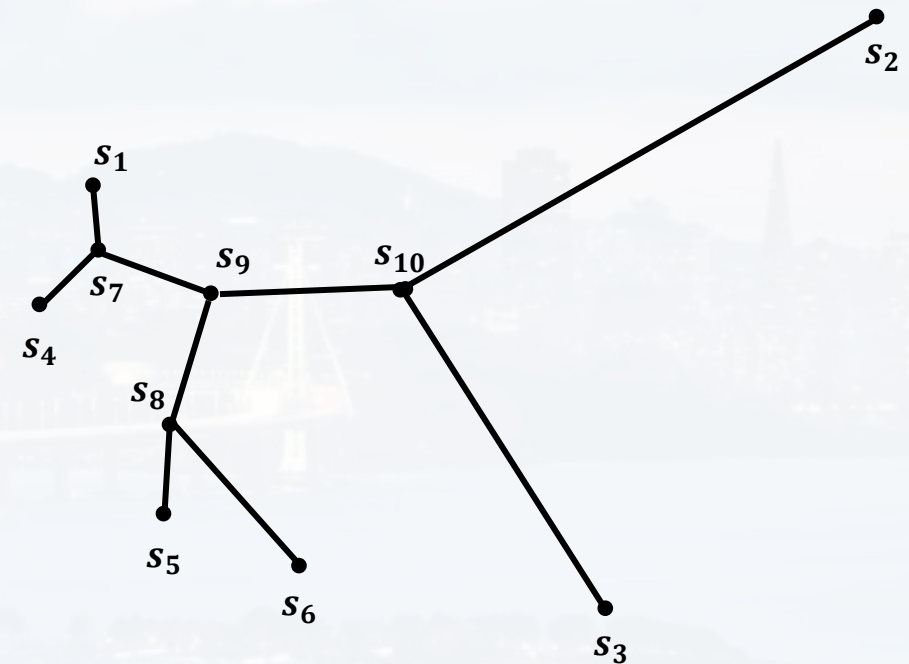
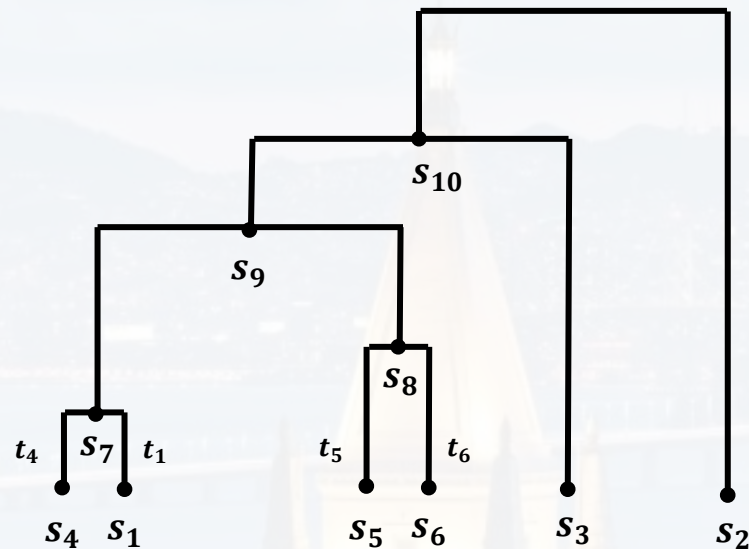
....finally

→ **Unweighted Pair Group Method**  
Using **Arithmetic Averages (UPGMA)**



constructing trees:

→ **Unweighted Pair Group Method**  
 Using **Arithmetic Averages (UPGMA)**



note: sometimes these diagrams might be misleading when interpreting the distances between the nodes

distances:

“A distance ***d*** is a function that assigns a **positive real number** to a set and/or to elements of a set.”

**properties:**

- $d(x, y) \geq 0$
- $d(x, y) = 0 \Leftrightarrow x = y$
- $d(x, y) = d(y, x)$
- $d(x, z) \leq d(x, y) + d(y, z)$

frequently used distances:

- Euclidean
- cityblock
- cosine
- correlation
- hamming
- Jukes Cantor

distances:

“A distance ***d*** is a function that assigns a **positive real number** to a set and/or to elements of a set.”

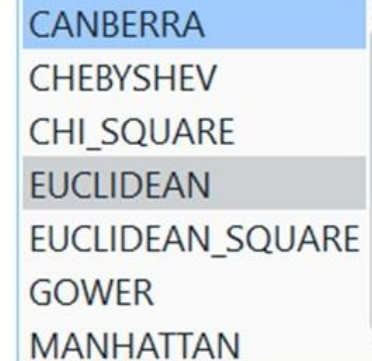
**properties:**

- $d(x, y) \geq 0$
- $d(x, y) = 0 \Leftrightarrow x = y$
- $d(x, y) = d(y, x)$
- $d(x, z) \leq d(x, y) + d(y, z)$

- Euclidean
- cityblock
- cosine
- correlation
- hamming
- Jukes Cantor

```
from pyclustering.utils.metric import *
```

type\_metric.



CANBERRA  
CHEBYSHEV  
CHI\_SQUARE  
EUCLIDEAN  
EUCLIDEAN\_SQUARE  
GOWER  
MANHATTAN

distances:

The Jukes-Cantor distance

**question: What could be a measure of distance here?**

Seq1 = {AAGTCTTTGAATCGTATGGGCGCGGCTTTCAAAA}

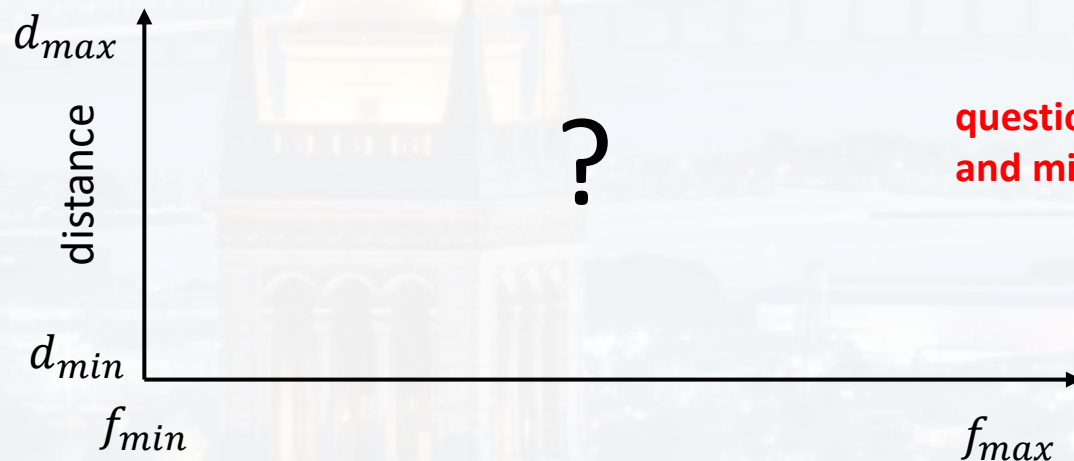
Seq2 = {AAGTCTATGAATCGTATGGGTCGGCTTTCAAAA}



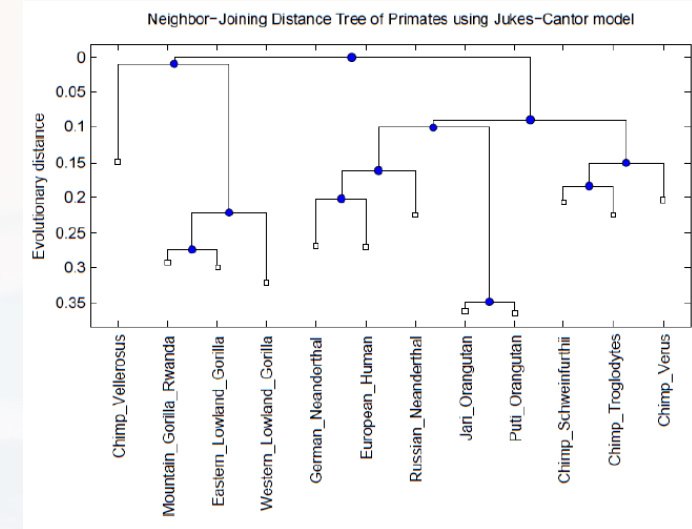
fraction  $f$  of different sites

if  $f$  is large  $\rightarrow d$  is large

if  $f$  is small  $\rightarrow d$  is small



**question: What are the maximum and minimum here?**



distances:

The Jukes-Cantor distance

$f$ : fraction of different sites

Seq1 = {AAGTCTTTGAATCGTATGGGCGCGGCTTTCAAAA}  
Seq2 = {AAGTCTATGAATCGTATGGGTCGGGCTTTCAAAA}

$f_{min} = 0 \rightarrow d_{min} = 0$  (sequences are identical)

for  $N$  letters ( $N = 4$  nucleotides,  $N = 20$  amino acids)

$$f_{max} = 1 - \frac{1}{N} \rightarrow d_{max} = \infty$$

(sequences have no relations,  
matches are coincidence)

- the probability that a site **has been** mutated, given a mutation rate  $\lambda$  equals:

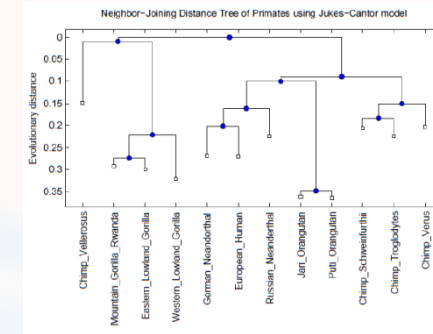
$$P(any|\lambda) = 1 - e^{-\lambda t}$$

- the probability that this mutation will lead to a **particular letter** is

$$P(part|\lambda) = \frac{1}{N} (1 - e^{-\lambda t})$$

- the probability that this mutation will lead to a **letter other than the previous one** is

$$P(part|\lambda) = \frac{N-1}{N} (1 - e^{-\lambda t})$$

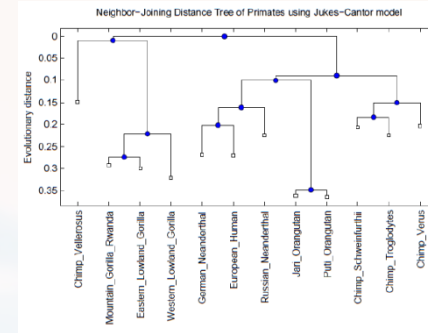


distances:

The Jukes-Cantor distance

$f$ : fraction of different sites

Seq1 = {AAGTCTTTGAATCGTATGGGCGCGGCTTTCAAAA}  
 Seq2 = {AAGTCTATGAATCGTATGGGTCGGCTTTCAAAA}



$f_{min} = 0 \rightarrow d_{min} = 0$  (sequences are identical)

for  $N$  letters ( $N = 4$  nucleotides,  $N = 20$  amino acids)

- the probability that this mutation will lead to a **letter other than the previous one** is

$$P(part|\lambda) = \frac{N-1}{N} (1 - e^{-\lambda t})$$

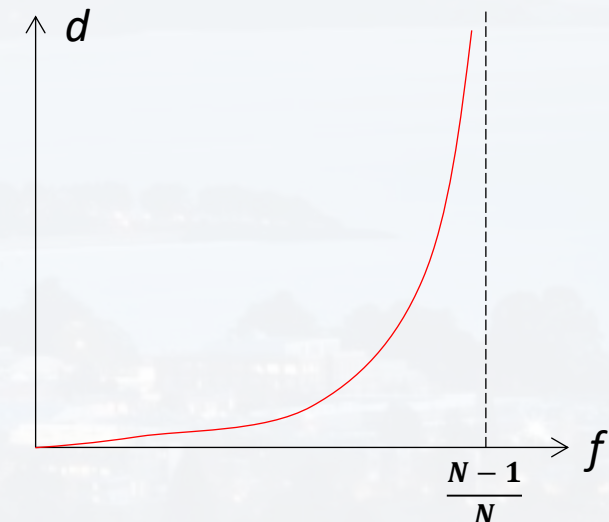
$\lambda t \sim d$  since cumulative mutations is what makes species different!

$$d \sim \lambda t = -\ln \left( 1 - \frac{N}{N-1} f \right)$$

often:

$$d = -\frac{N-1}{N} \ln \left( 1 - \frac{N}{N-1} f \right)$$

Jukes – Cantor distance





distances:

Jukes – Cantor distance 
$$d(x, \bar{x}) = -\frac{N-1}{N} \ln \left( 1 - \frac{N}{N-1} f(x, \bar{x}) \right)$$

Euclidean distance 
$$d(x, \bar{x})^2 = (x_1 - \bar{x}_1)^2 + (x_2 - \bar{x}_2)^2 \dots (x_N - \bar{x}_N)^2$$

cityblock 
$$d(x, \bar{x}) = |x_1 - \bar{x}_1| + |x_2 - \bar{x}_2| \dots |x_N - \bar{x}_N|$$

cosine 
$$d(x, \bar{x}) = 1 - \frac{x \cdot \bar{x}}{\sqrt{(x \cdot x)(\bar{x} \cdot \bar{x})}}$$

correlation 
$$d(x, \bar{x}) = 1 - \frac{\text{cov}(x, \bar{x})}{\sqrt{\text{var}(x)\text{var}(\bar{x})}}$$

...and many others

**note:** for sequence alignment and phylogenetic trees, chemical properties of the AA or NT have to be taken into account → **score matrices**

Python libraries:

libraries from the *Bio* package

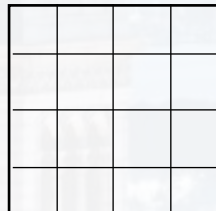
```
from Bio.Phylo.TreeConstruction import DistanceCalculator, DistanceTreeConstructor  
from Bio import Phylo  
from scipy import spatial
```

for plotting a  
phylogenetic tree

for rearranging a  
distance matrix

general idea:

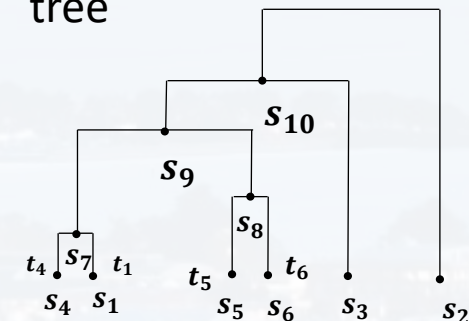
distance matrix



linkage algorithm (e.g. UPGMA)



tree



Python libraries:

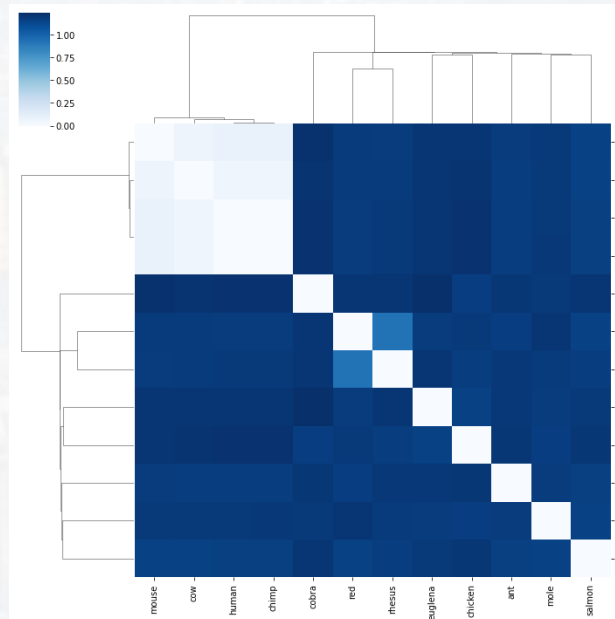
also most heatmap tools have some abilities to construct trees:

`sns.clustermap`

```
sns.clustermap(D_df, cmap = 'Blues', \
               row_cluster = True, col_cluster = True, \
               metric = 'euclidean', method = 'average', \
               yticklabels = True)
```

`plt.show()`

similar to UPGMA



topics for the discussion/office hour:

- distances
- random forest
- graphs

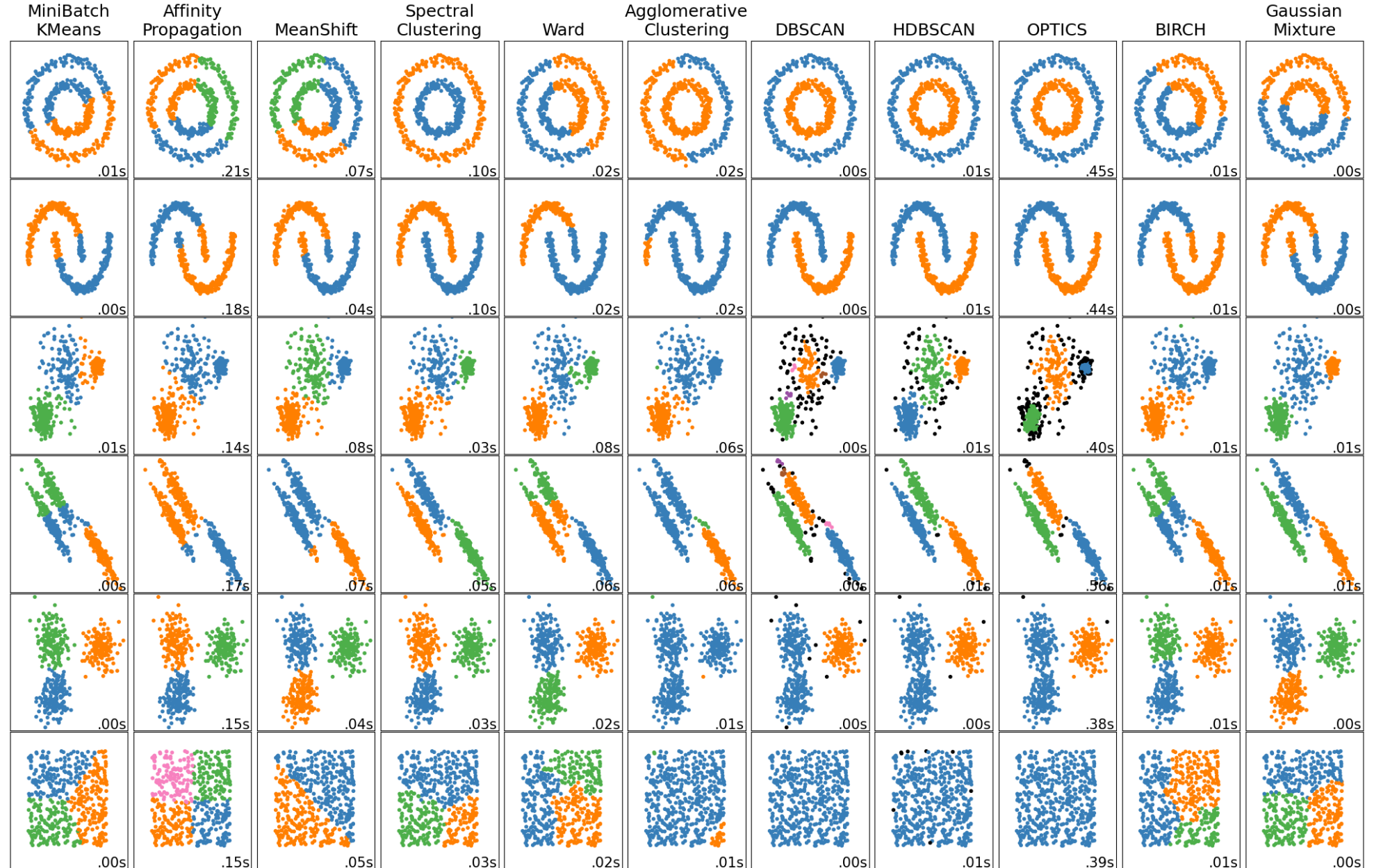
see also

**Walk\_Through\_Tree.ipynb**

for more details



there is a lot more...



Thank you very much for your attention!

